

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CNTT I



BÁO CÁO CUỐI KỲ

MÔN CÁC HỆ THỐNG PHÂN TÁN

**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG BÁN CÁ CẢNH FISHSHOP THEO
KIẾN TRÚC MICROSERVICE**

Giảng viên: Kim Ngọc Bách

Nhóm lớp: 01

Nhóm báo cáo: 08

Thành viên:

Họ tên	MSV
Đinh Ngọc Hân	B22DCCN282
Cao Đức Việt	B22DCCN894
Nguyễn Công Trung	B22DCCN870

Hà Nội, 2026

MỤC LỤC

MỤC LỤC	2
DANH MỤC HÌNH ẢNH	5
DANH MỤC BẢNG BIỂU	7
BẢNG PHÂN CHIA CÔNG VIỆC	8
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI	9
1.1. Lý do chọn đề tài	9
1.2. Mục tiêu đề tài	9
1.2.1. Mục tiêu nghiệp vụ	9
1.2.2. Mục tiêu môn học	10
1.3. Phạm vi đề tài	10
1.4. Đối tượng sử dụng	10
CHƯƠNG 2. PHÂN TÍCH YÊU CẦU HỆ THỐNG.....	12
2.1. Mô tả bài toán nghiệp vụ	12
2.2. Yêu cầu chức năng.....	13
2.2.1. Quản lý người dùng và xác thực	13
2.2.2. Quản lý sản phẩm và danh mục	13
2.2.3. Quản lý tồn kho.....	13
2.2.4. Quản lý giỏ hàng và đơn hàng	14
2.2.5. Quản lý khuyến mãi và thanh toán logic	15
2.2.6. Quản lý nội dung và liên hệ	15
2.3. Yêu cầu phi chức năng.....	16
2.4. Ràng buộc hệ thống	17
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG PHÂN TÁN.....	19
3.1. Thiết kế microservice.....	19

3.2. Kiến trúc tổng thể hệ thống	19
3.3. Thiết kế các microservice	21
3.3.1. User Service	21
3.3.2. Product Service	21
3.3.3. Order Service	22
3.3.4. Content Service	22
3.3.5. API Gateway	22
3.4. Thiết kế cơ sở dữ liệu	23
3.5. Thiết kế giao tiếp đồng bộ bằng RESTful API	25
3.6. Thiết kế giao tiếp bất đồng bộ bằng Message Queue và Saga	26
3.7. Thiết kế bảo mật	28
3.8. Thiết kế xử lý lỗi, logging và quan sát hệ thống	29
3.9. Tính nhất quán dữ liệu trong hệ thống	30
CHƯƠNG 4. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG	32
4.1. Công nghệ sử dụng	32
4.2. Cấu trúc mã nguồn	33
4.3. Cài đặt cơ sở dữ liệu	35
4.4. Triển khai backend và hạ tầng	36
4.5. Cấu hình backend quan trọng	38
4.5.1. Định tuyến qua API Gateway (YARP)	38
4.5.2. Chuỗi kết nối cơ sở dữ liệu	38
4.5.3. Cấu hình RabbitMQ và MassTransit	39
4.5.4. Biến môi trường và quan sát hệ thống	39
4.6. Triển khai frontend	40
4.6.1. Chạy frontend quản trị	40

4.6.2. Chạy frontend khách hàng	40
4.7. Giao diện hệ thống	40
4.7.1. Giao diện khách hàng	41
4.7.2. Giao diện quản trị	42
4.8. Kiểm tra nhanh sau triển khai	44
CHƯƠNG 5. KIỂM THỬ HỆ THỐNG	47
5.1. Mục tiêu kiểm thử	47
5.2. Môi trường kiểm thử	47
5.3. Phương pháp và công cụ kiểm thử	48
5.4. Các kịch bản kiểm thử	48
5.5. Nhận xét kết quả kiểm thử	54
CHƯƠNG 6. ĐÁNH GIÁ VÀ KẾT LUẬN	55
6.1. Kết quả đạt được	55
6.2. So sánh với kiến trúc nguyên khối	55
6.3. Ưu điểm	56
6.4. Hạn chế	56
6.5. Hướng phát triển	56
6.6. Kết luận	57
TÀI LIỆU THAM KHẢO	58

DANH MỤC HÌNH ẢNH

Hình 2.1. Quy trình nghiệp vụ tổng quát của hệ thống bán cá cảnh.....	13
Hình 2.2. Luồng nghiệp vụ đặt hàng của khách hàng	15
Hình 3.1. Kiến trúc tổng thể hệ thống FishShop	20
Hình 3.2. Luồng xử lý request từ client qua API Gateway	23
Hình 3.3. Phân chia cơ sở dữ liệu theo từng service (database-per-service).....	24
Hình 3.4. Sơ đồ ERD rút gọn của bốn cơ sở dữ liệu thành phần.....	25
Hình 3.5. Sơ đồ tuần tự Saga điều phối luồng đặt hàng	27
Hình 3.6. Sơ đồ producer – hàng đợi (queue) – consumer trên RabbitMQ.....	28
Hình 3.7. Luồng xác thực JWT và bổ sung header ngữ cảnh qua API Gateway	29
Hình 3.8. Luồng nhất quán dữ liệu và xử lý bù (compensation) trong Saga.....	31
Hình 4.1. Cấu trúc thư mục chính của dự án	34
Hình 4.2. Trang danh sách sản phẩm phía khách hàng	41
Hình 4.3. Trang giỏ hàng và thanh toán (checkout)	42
Hình 4.4. Trang theo dõi đơn hàng của khách hàng.....	42
Hình 4.5. Trang tổng quan (dashboard) quản trị	43
Hình 4.6. Màn hình quản lý sản phẩm.....	43
Hình 4.7. Màn hình quản lý đơn hàng	44
Hình 4.8. Màn hình quản lý blog và liên hệ	44
Hình 4.9. Kết quả gọi health-check các service qua API Gateway	45
Hình 4.10. Kết quả gọi API danh sách sản phẩm qua API Gateway.....	46
Hình 4.11. Trace trên Jaeger sau khi tạo đơn hàng.....	46
Hình 4.12. Dashboard Grafana sau khi tạo đơn hàng.....	46
Hình 5.1. Kết quả đăng nhập trên Postman	51
Hình 5.2. Kết quả tạo đơn hàng thành công	52

Hình 5.3. Các hàng đợi và message Saga trên RabbitMQ Management.....	53
Hình 5.4. Log Saga trên Seq và trace trên Jaeger	54

DANH MỤC BẢNG BIỂU

Bảng 2.1. Tổng hợp yêu cầu chức năng của hệ thống	16
Bảng 3.1. Phân chia cơ sở dữ liệu theo từng service	23
Bảng 3.2. Tổng hợp nhóm endpoint RESTful API theo service	26
Bảng 4.1. Công nghệ sử dụng trong hệ thống	32
Bảng 4.2. Danh sách cơ sở dữ liệu theo service	35
Bảng 4.3. Tài khoản mẫu phục vụ kiểm thử và demo	35
Bảng 4.4. Danh sách container backend và hạ tầng triển khai	36
Bảng 4.5. Địa chỉ truy cập các thành phần sau triển khai.....	37
Bảng 5.1. Môi trường và công cụ kiểm thử.....	47
Bảng 5.2. Tổng hợp các kịch bản kiểm thử chính	48

BẢNG PHÂN CHIA CÔNG VIỆC

STT	Họ và tên	Phân chia công việc
1	Cao Đức Việt	Order Service, RabbitMQ/Saga và cơ sở dữ liệu
2	Đinh Ngọc Hân	Kiến trúc hệ thống, User Service, Product Service và API Gateway
3	Nguyễn Công Trung	Frontend, Content Service, Logging/Monitoring

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong bối cảnh thương mại điện tử phát triển mạnh, các hệ thống bán hàng trực tuyến cần đáp ứng nhiều nghiệp vụ như quản lý sản phẩm, khách hàng, tồn kho, giỏ hàng, đơn hàng và nội dung tư vấn. Đối với cửa hàng cá cảnh, các nghiệp vụ này có liên hệ với nhau nhưng lại có đặc điểm xử lý khác nhau. Ví dụ, khách hàng cần tra cứu sản phẩm nhanh chóng, trong khi quá trình đặt hàng, giữ hàng, thanh toán logic và cập nhật tồn kho cần đảm bảo tính chính xác và khả năng phối hợp giữa nhiều thành phần.

Nếu xây dựng hệ thống theo mô hình nguyên khối, toàn bộ chức năng sẽ được đặt trong một ứng dụng duy nhất. Cách tiếp cận này đơn giản ở giai đoạn đầu nhưng gây khó khăn khi mở rộng, bảo trì hoặc thay đổi từng nhóm chức năng. Khi một thành phần gặp lỗi hoặc cần nâng cấp, toàn bộ hệ thống có thể bị ảnh hưởng.

Vì vậy, nhóm lựa chọn đề tài “Xây dựng hệ thống bán cá cảnh theo kiến trúc Microservice”. Kiến trúc này cho phép chia hệ thống thành nhiều dịch vụ độc lập, mỗi dịch vụ phụ trách một miền nghiệp vụ riêng và phối hợp với nhau thông qua cơ chế giao tiếp qua mạng. Cách tổ chức này phù hợp với đặc trưng của hệ thống phân tán, trong đó các thành phần độc lập cùng phối hợp để cung cấp một hệ thống thống nhất cho người sử dụng.

1.2. Mục tiêu đề tài

1.2.1. Mục tiêu nghiệp vụ

Mục tiêu nghiệp vụ của đề tài là xây dựng một hệ thống hỗ trợ hoạt động bán cá cảnh trực tuyến. Hệ thống cho phép khách hàng xem sản phẩm, thêm sản phẩm vào giỏ hàng, đặt hàng và theo dõi đơn hàng. Đồng thời, hệ thống hỗ trợ quản trị viên hoặc nhân viên quản lý sản phẩm, danh mục, tồn kho, đơn hàng, khách hàng và nội dung hiển thị trên website.

Thông qua các chức năng trên, hệ thống thực hiện được quy trình bán hàng cơ bản của một cửa hàng cá cảnh trực tuyến, từ phía khách hàng mua hàng đến phía quản trị viên vận hành và xử lý đơn hàng.

1.2.2. Mục tiêu môn học

Về mặt môn học, đề tài hướng tới việc vận dụng kiến thức về hệ thống phân tán vào một bài toán thực tế. Hệ thống cần được tổ chức thành nhiều dịch vụ độc lập, có khả năng phối hợp với nhau để xử lý nghiệp vụ chung.

Cụ thể, hệ thống cần có giao tiếp đồng bộ thông qua RESTful API, giao tiếp bất đồng bộ thông qua Message Queue, dữ liệu được phân chia theo từng miền nghiệp vụ và có cơ chế xử lý lỗi, ghi log cơ bản. Ngoài yêu cầu tối thiểu, hệ thống hiện tại còn bổ sung Saga cho luồng đặt hàng, tracing/metrics bằng OpenTelemetry và các công cụ quan sát như Jaeger, Prometheus, Grafana nhằm thể hiện rõ hơn đặc trưng của hệ thống phân tán.

1.3. Phạm vi đề tài

Trong phạm vi đồ án môn học, hệ thống tập trung vào các nghiệp vụ cốt lõi của một cửa hàng cá cảnh trực tuyến. Các chức năng chính bao gồm quản lý người dùng, quản lý sản phẩm, danh mục, tồn kho, giỏ hàng, đơn hàng, khuyến mãi, thanh toán logic và quản lý nội dung.

Trọng tâm của đề tài là xây dựng được hệ thống theo kiến trúc microservice và demo được luồng nghiệp vụ có sự phối hợp giữa nhiều dịch vụ. Trong đó, luồng đặt hàng, giữ tồn kho, xử lý thanh toán mô phỏng và cập nhật trạng thái đơn hàng được xem là luồng quan trọng nhất để thể hiện đặc điểm của hệ thống phân tán.

Hệ thống hướng tới mục tiêu học tập và demo kiến trúc, do đó phạm vi triển khai tập trung vào các chức năng cốt lõi và các cơ chế kỹ thuật tiêu biểu như database-per-service, event-driven workflow, logging tập trung, tracing và monitoring. Các nội dung như tích hợp cổng thanh toán thật, kiểm thử tải lớn, triển khai production trên cloud hoặc Kubernetes được xem là hướng mở rộng trong các giai đoạn phát triển tiếp theo.

1.4. Đối tượng sử dụng

Hệ thống hướng tới ba nhóm đối tượng chính: khách hàng, quản trị viên/nhân viên và các dịch vụ nội bộ trong hệ thống.

- **Khách hàng:** người sử dụng giao diện mua hàng của hệ thống. Nhóm người dùng này có nhu cầu xem thông tin sản phẩm cá cảnh, tìm kiếm sản phẩm phù hợp, thêm sản phẩm vào giỏ hàng, đặt hàng và theo dõi trạng thái đơn hàng.
- **Quản trị viên hoặc nhân viên:** nhóm người dùng vận hành hệ thống. Nhóm này thực hiện các thao tác quản lý sản phẩm, danh mục, tồn kho, đơn hàng, khách hàng và nội dung.
- **Các dịch vụ nội bộ:** các thành phần backend trong kiến trúc microservice. Mỗi dịch vụ phụ trách một miền nghiệp vụ riêng và phối hợp với các dịch vụ khác thông qua RESTful API hoặc Message Queue.

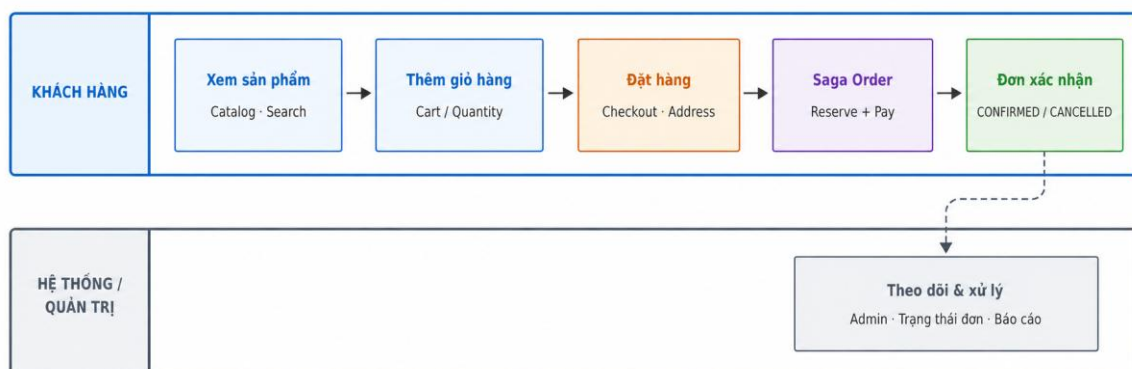
CHƯƠNG 2. PHÂN TÍCH YÊU CẦU HỆ THỐNG

2.1. Mô tả bài toán nghiệp vụ

Hệ thống FishShop được xây dựng nhằm hỗ trợ hoạt động bán cá cảnh trực tuyến, bao gồm hai nhóm nghiệp vụ chính: nghiệp vụ mua hàng của khách hàng và nghiệp vụ quản trị, vận hành của cửa hàng. Quy trình tổng quát của hệ thống có thể mô tả như sau:

- **Khách hàng truy cập hệ thống:** để xem danh sách sản phẩm cá cảnh, tra cứu thông tin chi tiết, giá bán, hình ảnh và mô tả sản phẩm.
- **Khách hàng thêm sản phẩm vào giỏ hàng:** sau khi lựa chọn được sản phẩm phù hợp, đồng thời có thể điều chỉnh số lượng hoặc xóa sản phẩm khỏi giỏ.
- **Khách hàng tiến hành đặt hàng:** hệ thống ghi nhận thông tin đơn hàng, danh sách sản phẩm, số lượng, tổng tiền và trạng thái xử lý ban đầu.
- **Hệ thống xử lý tồn kho và thanh toán logic:** thông qua luồng bất đồng bộ để giữ hàng, xử lý thanh toán mô phỏng, hoàn tất đơn hàng hoặc thực hiện bù trừ khi có lỗi.
- **Quản trị viên hoặc nhân viên cửa hàng:** theo dõi đơn hàng, cập nhật trạng thái xử lý, quản lý sản phẩm, danh mục, tồn kho, khách hàng và nội dung hiển thị trên website.

Luồng đặt hàng, giữ tồn kho, xử lý thanh toán mô phỏng và cập nhật trạng thái đơn hàng là nghiệp vụ trọng tâm của hệ thống vì có sự phối hợp giữa nhiều thành phần xử lý. Đây cũng là cơ sở để thể hiện đặc trưng của hệ thống phân tán trong đề tài.



Hình 2.1. Quy trình nghiệp vụ tổng quát của hệ thống bán cá cảnh

2.2. Yêu cầu chức năng

Hệ thống cần đáp ứng các nhóm chức năng chính phục vụ cả phía khách hàng và phía quản trị viên. Các chức năng được phân chia theo từng nhóm nghiệp vụ để thuận tiện cho việc thiết kế, triển khai và kiểm thử.

2.2.1. Quản lý người dùng và xác thực

Hệ thống cần hỗ trợ người dùng đăng ký, đăng nhập và truy cập các chức năng phù hợp với quyền của mình. Các yêu cầu chính gồm:

- Cho phép khách hàng đăng ký tài khoản và đăng nhập vào hệ thống.
- Cho phép người dùng xem và cập nhật thông tin cá nhân.
- Hỗ trợ phân biệt quyền giữa khách hàng và quản trị viên.
- Chỉ cho phép tài khoản có quyền phù hợp truy cập các chức năng quản trị.
- Đảm bảo các chức năng như giỏ hàng, đặt hàng và xem đơn hàng cá nhân chỉ được sử dụng sau khi người dùng đã xác thực.

2.2.2. Quản lý sản phẩm và danh mục

Sản phẩm là dữ liệu trung tâm của hệ thống bán cá cảnh, cần được hiển thị rõ ràng cho khách hàng và quản lý đầy đủ ở phía quản trị. Các yêu cầu chính gồm:

- Hiển thị danh sách sản phẩm cá cảnh cho khách hàng.
- Cho phép khách hàng xem thông tin chi tiết của từng sản phẩm.
- Hỗ trợ tìm kiếm, phân trang hoặc lọc sản phẩm để tăng trải nghiệm tra cứu.
- Cho phép quản trị viên thêm, sửa, xóa và cập nhật trạng thái sản phẩm.
- Cho phép quản trị viên quản lý danh mục sản phẩm để phân loại sản phẩm trong hệ thống.

2.2.3. Quản lý tồn kho

Tồn kho cần được quản lý để đảm bảo hệ thống phản ánh đúng số lượng sản phẩm hiện có và hỗ trợ quá trình xử lý đơn hàng. Các yêu cầu chính gồm:

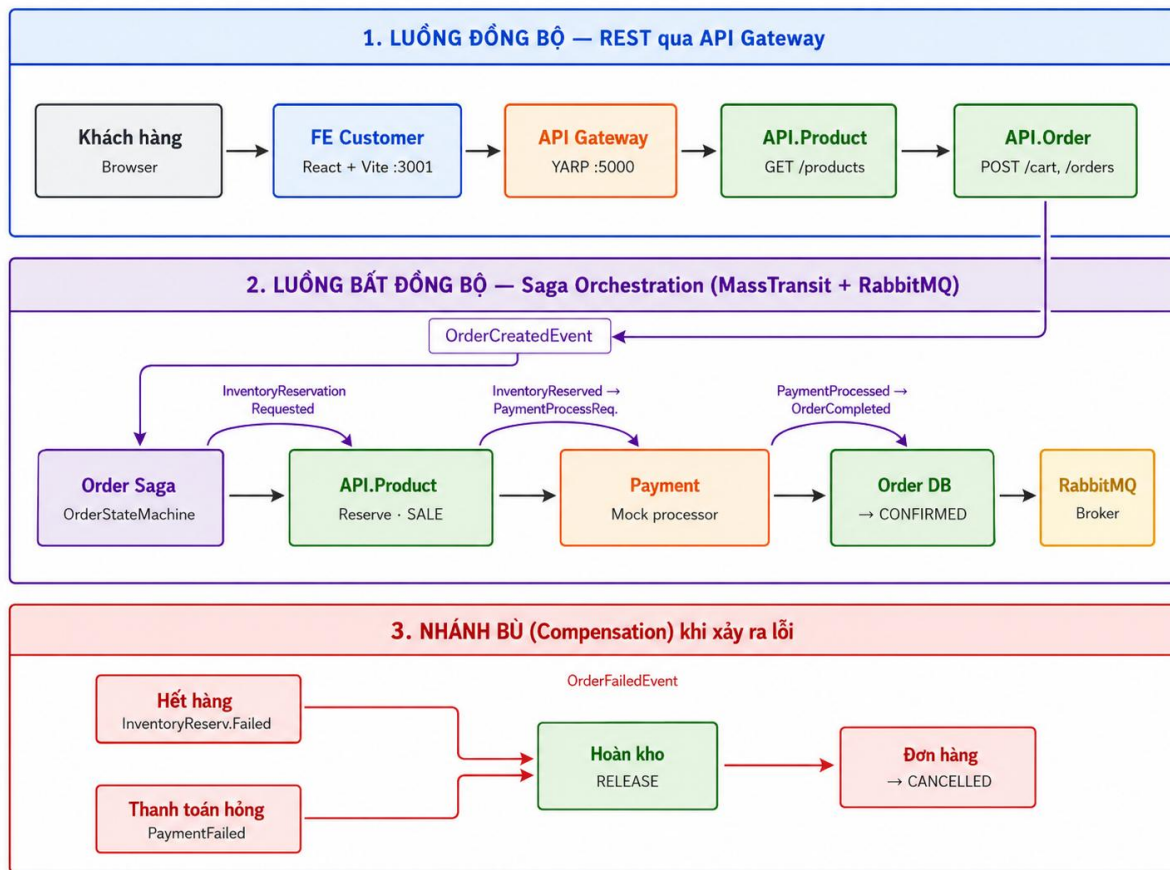
- Lưu trữ số lượng tồn của từng sản phẩm.

- Cho phép quản trị viên ghi nhận hoạt động nhập kho.
- Cập nhật số lượng tồn khi phát sinh giao dịch xuất kho, giữ hàng hoặc hoàn tồn kho.
- Ghi nhận lịch sử giao dịch kho để phục vụ kiểm tra, đối chiếu.
- Đảm bảo tồn kho được cập nhật phù hợp theo kết quả của luồng đặt hàng và xử lý bù.

2.2.4. Quản lý giỏ hàng và đơn hàng

Giỏ hàng và đơn hàng là nhóm chức năng quan trọng nhất trong quy trình mua hàng của khách hàng. Các yêu cầu chính gồm:

- Cho phép khách hàng thêm sản phẩm vào giỏ hàng.
- Cho phép cập nhật số lượng hoặc xóa sản phẩm khỏi giỏ hàng.
- Hiển thị thông tin giỏ hàng trước khi khách hàng xác nhận đặt hàng.
- Tạo đơn hàng từ thông tin trong giỏ hàng hoặc từ sản phẩm được chọn.
- Lưu thông tin đơn hàng gồm khách hàng, sản phẩm, số lượng, tổng tiền và trạng thái.
- Cập nhật trạng thái đơn hàng theo kết quả của Saga, ví dụ hoàn tất đơn hoặc hủy đơn khi giữ kho/thanh toán mô phỏng thất bại.
- Cho phép khách hàng xem danh sách đơn hàng của mình.
- Cho phép quản trị viên xem chi tiết đơn hàng và cập nhật trạng thái xử lý khi phù hợp.



Hình 2.2. Luồng nghiệp vụ đặt hàng của khách hàng

2.2.5. Quản lý khuyến mãi và thanh toán logic

Trong phạm vi đề tài, hệ thống hỗ trợ khuyến mãi và thanh toán ở mức mô phỏng nghiệp vụ, chưa tích hợp công thanh toán thật. Các yêu cầu chính gồm:

- Cho phép quản trị viên tạo và quản lý mã khuyến mãi.
- Cho phép khách hàng áp dụng mã khuyến mãi hợp lệ khi đặt hàng.
- Tính toán giá trị giảm giá vào tổng tiền đơn hàng.
- Ghi nhận phương thức thanh toán và trạng thái thanh toán ở mức logic.
- Hỗ trợ mô phỏng kết quả thanh toán trong luồng Saga để hoàn tất đơn hoặc kích hoạt xử lý bù.

2.2.6. Quản lý nội dung và liên hệ

Ngoài chức năng bán hàng, hệ thống cần có phần nội dung để cung cấp thông tin tư vấn và tăng tương tác với khách hàng. Các yêu cầu chính gồm:

- Hiển thị danh sách bài viết hoặc nội dung tư vấn cho khách hàng.
- Cho phép khách hàng xem chi tiết bài viết.
- Cho phép quản trị viên quản lý bài viết và danh mục bài viết.
- Cho phép khách hàng gửi thông tin liên hệ hoặc phản hồi.
- Cho phép quản trị viên xem và cập nhật trạng thái xử lý liên hệ.

Bảng 2.1. Tổng hợp yêu cầu chức năng của hệ thống

Nhóm chức năng	Người sử dụng chính	Mô tả ngắn
Người dùng và xác thực	Khách hàng, quản trị viên	Đăng ký, đăng nhập, phân quyền
Sản phẩm và danh mục	Khách hàng, quản trị viên	Xem, tìm kiếm, quản lý sản phẩm
Tồn kho	Quản trị viên, hệ thống	Theo dõi, giữ hàng, hoàn hàng và cập nhật tồn kho
Giỏ hàng và đơn hàng	Khách hàng, quản trị viên	Tạo đơn, theo dõi, cập nhật trạng thái
Khuyến mãi và thanh toán logic	Khách hàng, quản trị viên	Áp dụng mã, ghi nhận thanh toán mô phỏng
Nội dung và liên hệ	Khách hàng, quản trị viên	Blog, liên hệ, xử lý phản hồi

2.3. Yêu cầu phi chức năng

Bên cạnh các chức năng nghiệp vụ, hệ thống cần đáp ứng một số yêu cầu phi chức năng để phù hợp với định hướng xây dựng hệ thống phân tán. Các yêu cầu này giúp hệ thống có khả năng mở rộng, dễ bảo trì, an toàn hơn và thuận tiện trong quá trình vận hành.

- **Khả năng mở rộng:** hệ thống cần có khả năng mở rộng theo từng nhóm nghiệp vụ, thay vì phải mở rộng toàn bộ ứng dụng.

- **Tính sẵn sàng:** các chức năng chính cần duy trì khả năng phục vụ người dùng; lỗi ở một chức năng phụ không nên làm gián đoạn toàn bộ hệ thống.
- **Hiệu năng phản hồi:** các thao tác thường xuyên như xem sản phẩm, thêm giỏ hàng và tạo đơn hàng cần có thời gian phản hồi phù hợp.
- **Tính nhất quán dữ liệu:** dữ liệu cần đúng về mặt nghiệp vụ, đặc biệt trong luồng tạo đơn, giữ tồn kho, xử lý thanh toán mô phỏng và bù trừ.
- **Khả năng chịu lỗi:** hệ thống cần có xử lý lỗi cơ bản, retry cho message consumer và cơ chế retry/timeout/circuit breaker cho các HTTP call liên service.
- **Bảo mật và phân quyền:** hệ thống cần xác thực người dùng, phân quyền khách hàng/quản trị viên và kiểm soát request nội bộ bằng API key ở các API cần bảo vệ.
- **Khả năng quan sát:** hệ thống cần ghi log tập trung, tracing và metrics để theo dõi request, message, lỗi xử lý và hiệu năng service.
- **Dễ triển khai và bảo trì:** các thành phần cần được tổ chức rõ ràng, tách trách nhiệm hợp lý để thuận tiện cho việc chạy thử, kiểm thử, sửa đổi và mở rộng.

2.4. Ràng buộc hệ thống

Hệ thống được xây dựng trong phạm vi đồ án môn học nên ưu tiên khả năng demo, kiểm thử và thể hiện đúng kiến thức hệ thống phân tán. Các chức năng được triển khai tập trung vào nghiệp vụ bán cá cảnh trực tuyến và luồng xử lý liên dịch vụ, chưa hướng tới đầy đủ các yêu cầu của một hệ thống thương mại điện tử production.

- Hệ thống chạy chủ yếu trong môi trường học tập và demo bằng Docker Compose.
- Dữ liệu sử dụng trong hệ thống là dữ liệu mẫu phục vụ kiểm thử và trình bày.
- Chức năng thanh toán mới dừng ở mức mô phỏng logic, chưa tích hợp cổng thanh toán thật.
- Hệ thống chưa thực hiện kiểm thử tải lớn với số lượng người dùng cao.
- Hệ thống chưa triển khai trên cloud hoặc Kubernetes.
- Các cơ chế Saga, tracing, metrics và resilience được triển khai ở mức phục vụ đồ án, chưa thay thế đầy đủ các cơ chế vận hành production.

Với các ràng buộc trên, mục tiêu trọng tâm của hệ thống là chứng minh được kiến trúc microservice, khả năng phối hợp giữa các dịch vụ và luồng nghiệp vụ phân tán chính: khách hàng đặt hàng, hệ thống ghi nhận đơn hàng, giữ tồn kho, xử lý thanh toán mô phỏng và cập nhật trạng thái đơn hàng tương ứng.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG PHÂN TÁN

3.1. Thiết kế microservice

Hệ thống FishShop được thiết kế theo kiến trúc microservice nhằm phân tách các nhóm nghiệp vụ chính thành các dịch vụ độc lập. Mỗi dịch vụ phụ trách một miền nghiệp vụ riêng, có dữ liệu và logic xử lý riêng, đồng thời phối hợp với các dịch vụ khác thông qua cơ chế giao tiếp qua mạng.

Việc lựa chọn kiến trúc microservice phù hợp với đặc điểm của hệ thống phân tán. Người dùng bên ngoài vẫn truy cập hệ thống như một ứng dụng thống nhất, nhưng bên trong các chức năng được xử lý bởi nhiều dịch vụ khác nhau, có giao tiếp đồng bộ bằng REST và giao tiếp bất đồng bộ bằng RabbitMQ.

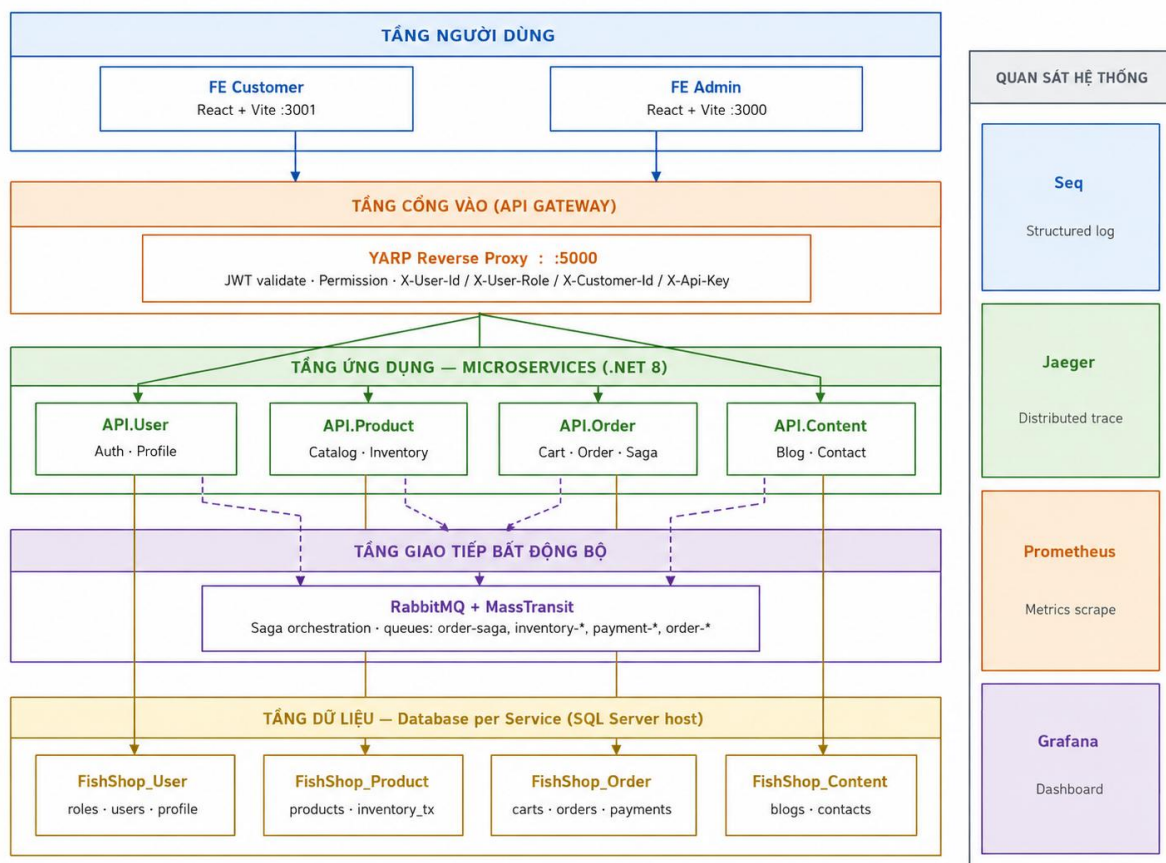
- **Tách dịch vụ theo miền nghiệp vụ:** các nghiệp vụ người dùng, sản phẩm, đơn hàng và nội dung được tách thành các service riêng để giảm phụ thuộc và dễ bảo trì.
- **Giao tiếp qua API và thông điệp:** các service giao tiếp đồng bộ bằng RESTful API và giao tiếp bất đồng bộ bằng Message Queue trong các luồng cần tách xử lý.
- **Dữ liệu tách theo service:** mỗi service quản lý vùng dữ liệu riêng, không truy cập trực tiếp database của service khác.
- **Có cổng truy cập chung:** client gọi hệ thống thông qua API Gateway để định tuyến request, kiểm soát xác thực và che giấu cấu trúc nội bộ.
- **Saga và xử lý bù:** luồng đặt hàng sử dụng Saga orchestration để phối hợp giữa đặt đơn, giữ tồn kho, thanh toán mô phỏng và cập nhật trạng thái đơn.
- **Quan sát và độ bền:** hệ thống sử dụng logging tập trung, tracing, metrics, retry, timeout và circuit breaker để hỗ trợ kiểm thử, debug và vận hành.

3.2. Kiến trúc tổng thể hệ thống

Hệ thống được tổ chức theo mô hình nhiều lớp, bao gồm tầng giao diện, tầng Gateway, tầng service nghiệp vụ, tầng dữ liệu, tầng giao tiếp bất đồng bộ và tầng quan sát hệ thống.

Tầng giao diện gồm hai ứng dụng frontend: giao diện khách hàng và giao diện quản trị. Người dùng thao tác trên frontend, các request được gửi đến API Gateway. Gateway là điểm vào chung của hệ thống, có nhiệm vụ định tuyến request đến các service phù hợp, kiểm tra JWT cho các route protected và bổ sung header ngữ cảnh người dùng cho request nội bộ.

Bên dưới Gateway là bốn microservice nghiệp vụ gồm API.User, API.Product, API.Order và API.Content. Mỗi service quản lý một nhóm chức năng riêng và kết nối với cơ sở dữ liệu SQL Server tương ứng. Ngoài RESTful API, hệ thống sử dụng RabbitMQ/MassTransit để triển khai luồng bất đồng bộ và Saga cho quá trình đặt hàng. Hệ thống cũng có Serilog + Seq cho logging tập trung, OpenTelemetry + Jaeger cho tracing, Prometheus + Grafana cho metrics và dashboard.



Hình 3.1. Kiến trúc tổng thể hệ thống FishShop

Kiến trúc này giúp hệ thống đảm bảo tính phân tách trách nhiệm. Mỗi service có thể được phát triển và kiểm thử theo phạm vi nghiệp vụ riêng. Đồng thời, Gateway giúp

người dùng bên ngoài không cần biết hệ thống bên trong có bao nhiêu service, từ đó tạo cảm giác hệ thống hoạt động như một ứng dụng thống nhất.

3.3. Thiết kế các microservice

3.3.1. User Service

User Service phụ trách các chức năng liên quan đến người dùng, xác thực và quản lý khách hàng. Đây là service cung cấp thông tin định danh người dùng cho các phần còn lại của hệ thống.

- Xử lý đăng ký, đăng nhập và làm mới token.
- Quản lý thông tin tài khoản và hồ sơ khách hàng.
- Cung cấp thông tin người dùng hiện tại cho các request đã xác thực.
- Hỗ trợ phân quyền cơ bản giữa khách hàng và quản trị viên.
- Cung cấp thông tin người dùng cho Order Service khi cần bổ sung dữ liệu đơn hàng.

3.3.2. Product Service

Product Service phụ trách các chức năng liên quan đến sản phẩm, danh mục, ảnh sản phẩm và tồn kho. Đây là service quản lý dữ liệu sản phẩm cá cảnh, phục vụ cả phía khách hàng và phía quản trị viên.

- Quản lý danh sách sản phẩm cá cảnh, danh mục sản phẩm và ảnh sản phẩm.
- Quản lý số lượng tồn kho và lịch sử giao dịch kho.
- Cung cấp thông tin sản phẩm, giá bán và số lượng tồn cho Order Service khi tạo giỏ hàng hoặc đơn hàng.
- Consume sự kiện InventoryReservationRequested để giữ/trừ tồn kho khi Saga yêu cầu.
- Consume sự kiện InventoryReleaseRequested để hoàn tồn kho khi thanh toán mô phỏng thất bại hoặc luồng xử lý cần bù trừ.
- Publish các sự kiện InventoryReservedEvent, InventoryReservationFailedEvent và InventoryReleasedEvent để Saga tiếp tục điều phối luồng xử lý.

3.3.3. Order Service

Order Service phụ trách các chức năng liên quan đến giỏ hàng, đơn hàng, thanh toán logic, khuyến mãi và báo cáo. Đây là service xử lý nghiệp vụ trung tâm của quy trình mua hàng và cũng là nơi điều phối Saga trong luồng đặt hàng.

- Quản lý giỏ hàng của khách hàng.
- Tạo đơn hàng từ giỏ hàng hoặc từ sản phẩm được chọn.
- Lưu thông tin đơn hàng, sản phẩm trong đơn, tổng tiền và trạng thái đơn.
- Quản lý mã khuyến mãi và trạng thái thanh toán ở mức mô phỏng.
- Cung cấp dữ liệu báo cáo, dashboard phục vụ quản trị.
- Publish OrderCreatedEvent sau khi đơn hàng được ghi nhận thành công.
- Chạy Saga orchestration để điều phối giữ tồn kho, thanh toán mô phỏng, hoàn tồn kho khi lỗi và cập nhật trạng thái đơn hàng.

3.3.4. Content Service

Content Service phụ trách các chức năng liên quan đến nội dung hiển thị trên website, bao gồm blog, danh mục blog và liên hệ khách hàng.

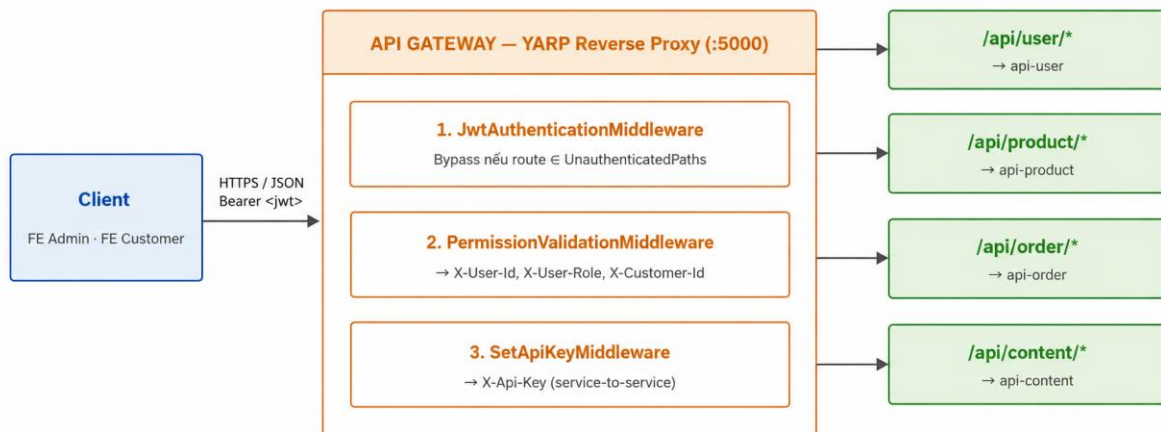
- Quản lý danh sách bài viết blog.
- Quản lý danh mục bài viết.
- Cung cấp API công khai để khách hàng xem bài viết.
- Tiếp nhận thông tin liên hệ từ khách hàng.
- Cho phép quản trị viên theo dõi và cập nhật trạng thái xử lý liên hệ.

3.3.5. API Gateway

API Gateway là điểm truy cập chung của toàn bộ backend. Thay vì để frontend gọi trực tiếp từng service, mọi request từ client đều đi qua Gateway. Gateway có nhiệm vụ định tuyến request, kiểm tra xác thực và bổ sung thông tin cần thiết trước khi chuyển tiếp request đến service đích.

- Định tuyến request theo đường dẫn `/api/{service}/...` đến service phù hợp.
- Kiểm tra JWT đối với các route cần xác thực.

- Cho phép bỏ qua xác thực với một số route public được khai báo trong UnauthenticatedPaths.
- Bổ sung header ngữ cảnh người dùng như X-User-Id, X-User-Role, X-Customer-Id vào request nội bộ.
- Bổ sung X-API-Key để các service phía sau xác nhận request đến từ Gateway.
- Che giấu cấu trúc service nội bộ đối với frontend.



Hình 3.2. Luồng xử lý request từ client qua API Gateway

3.4. Thiết kế cơ sở dữ liệu

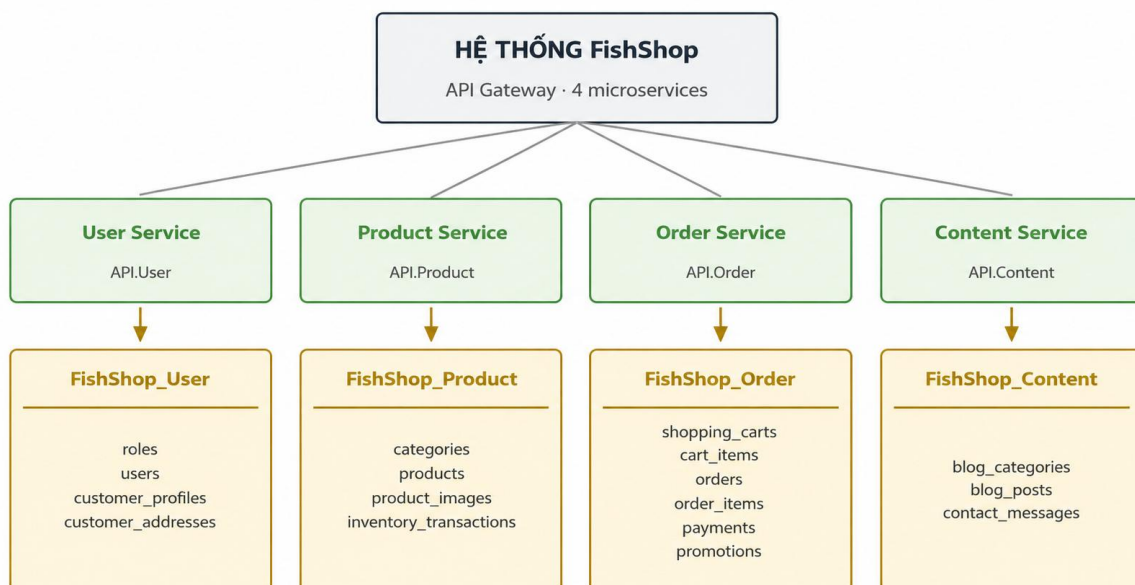
Hệ thống áp dụng nguyên tắc Database per Service, nghĩa là mỗi service sở hữu và quản lý cơ sở dữ liệu riêng. Cách tổ chức này giúp dữ liệu được phân chia theo miền nghiệp vụ, giảm phụ thuộc trực tiếp giữa các service và phù hợp với kiến trúc microservice.

Bảng 3.1. Phân chia cơ sở dữ liệu theo từng service

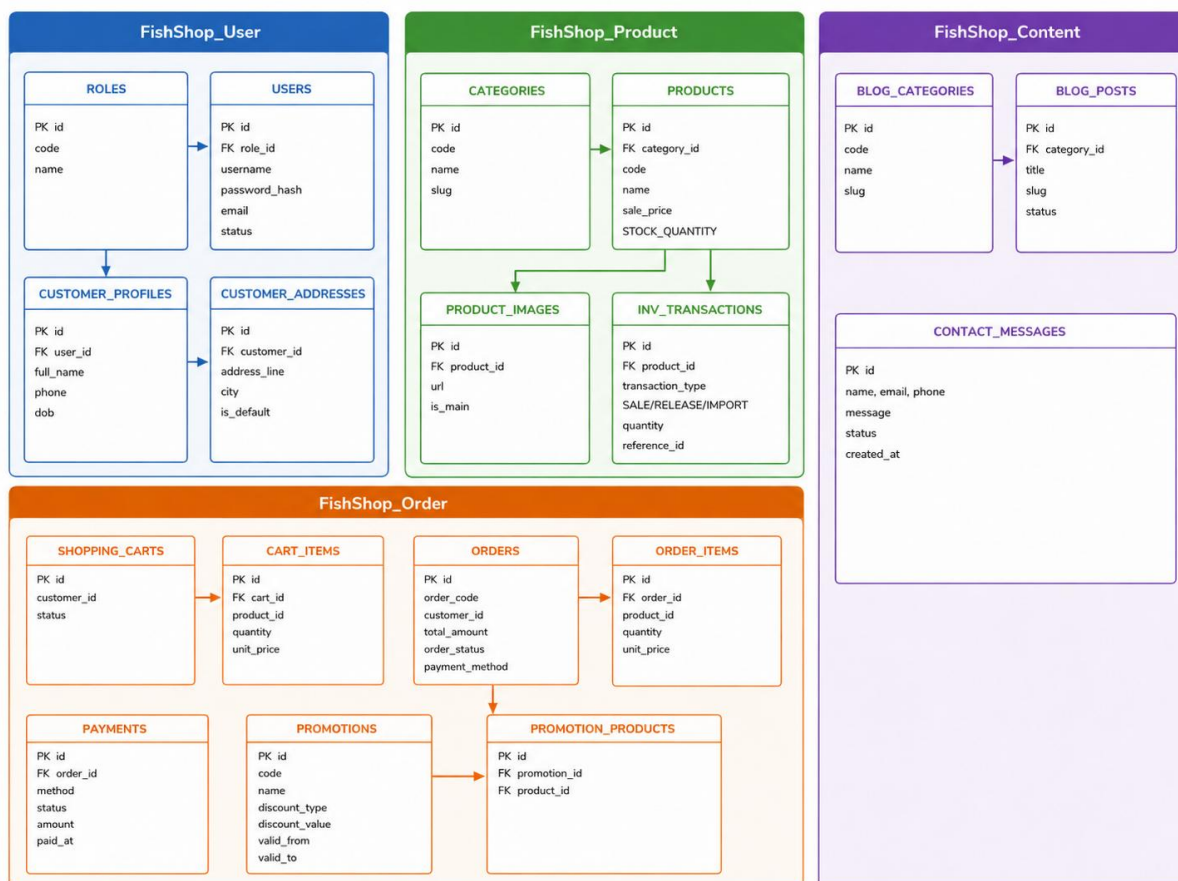
Service	Database	Nhóm dữ liệu chính
User Service	FishShop_User	roles, users, customer_profiles, customer_addresses

Product Service	FishShop_Product	categories, products, product_images, inventory, inventory_transactions
Order Service	FishShop_Order	shopping_carts, cart_items, orders, order_items, payments, promotions, promotion_products, promotion_usages
Content Service	FishShop_Content	blog_categories, blog_posts, contact_messages

Việc tách cơ sở dữ liệu giúp mỗi service có quyền kiểm soát dữ liệu của riêng mình. Các service khác không truy cập trực tiếp vào database của service đó, mà trao đổi thông qua API hoặc message. Ví dụ, Order Service không truy vấn trực tiếp bảng sản phẩm trong Product Database, mà gọi Product Service để lấy thông tin sản phẩm và kiểm tra tồn kho.



Hình 3.3. Phân chia cơ sở dữ liệu theo từng service (database-per-service)



Hình 3.4. Sơ đồ ERD rút gọn của bốn cơ sở dữ liệu thành phần

3.5. Thiết kế giao tiếp đồng bộ bằng RESTful API

Giao tiếp đồng bộ trong hệ thống được thực hiện thông qua RESTful API. Đây là hình thức giao tiếp phù hợp với các thao tác cần phản hồi trực tiếp, chẳng hạn như đăng nhập, xem sản phẩm, thêm giỏ hàng, tạo đơn hàng hoặc truy vấn dữ liệu quản trị.

Các request từ frontend được gửi đến Gateway theo đường dẫn bắt đầu bằng /api. Gateway định tuyến request đến service tương ứng dựa trên nhóm nghiệp vụ. Cách đặt route được tổ chức theo dạng /api/{service}/{resource}.

Bên cạnh việc frontend gọi API qua Gateway, một số service cũng cần gọi REST API đến service khác để lấy dữ liệu nghiệp vụ. Khi tạo đơn, Order Service gọi Product Service để lấy thông tin sản phẩm/tồn kho và có thể gọi User Service để lấy thông tin người dùng khi cần. Các HTTP call liên service được bổ sung resilience handler như retry, timeout và circuit breaker để tăng khả năng chịu lỗi trong trường hợp lỗi tạm thời.

Bảng 3.2. Tổng hợp nhóm endpoint RESTful API theo service

Service	Nhóm endpoint	Chức năng
User Service	/auth, /customers	Xác thực, hồ sơ, quản lý khách hàng
Product Service	/products, /categories, /inventory, /product-images	Sản phẩm, danh mục, tồn kho/ lịch sử kho và ảnh sản phẩm.
Order Service	/cart, /orders, /payments, /promotions, /reports, /sales, /dashboard	Giỏ hàng, đơn hàng, thanh toán, khuyến mãi, báo cáo, thống kê, dashboard
Content Service	/blogs, /blog-categories, /contacts	Blog, danh mục blog, liên hệ

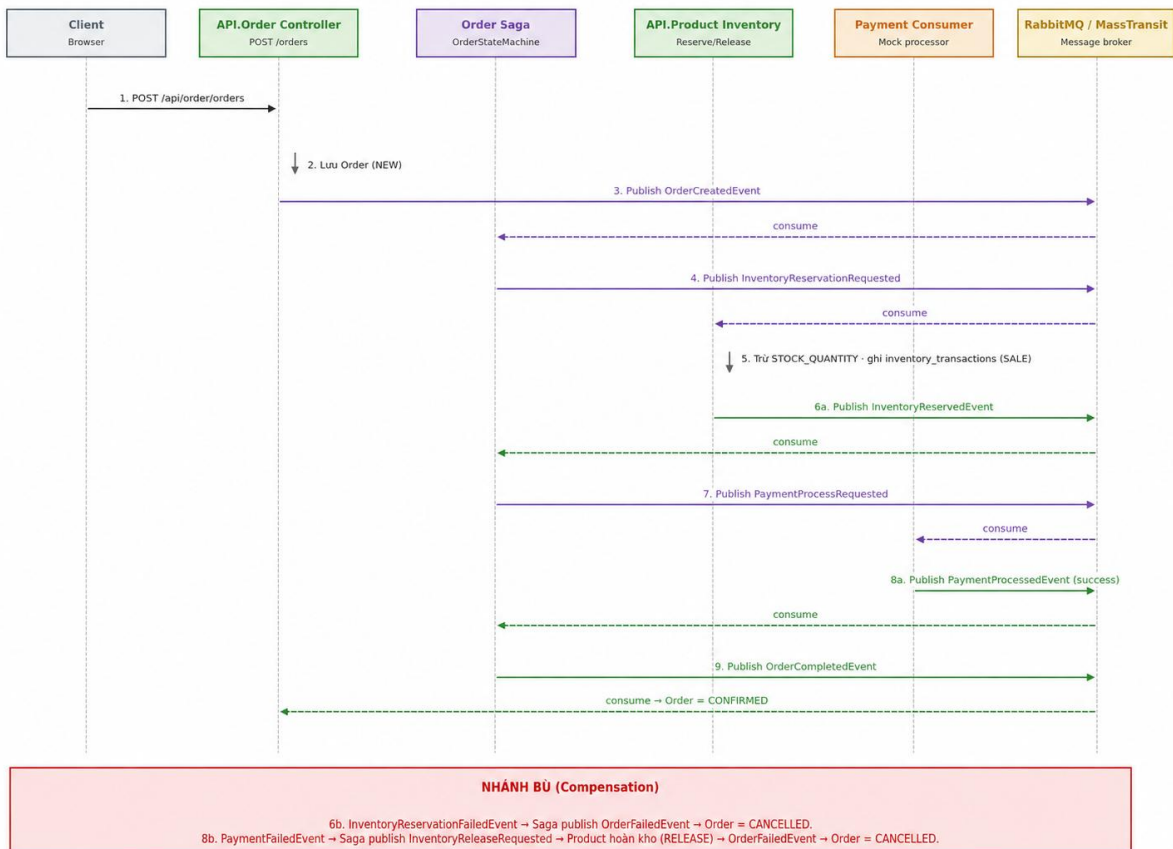
3.6. Thiết kế giao tiếp bất đồng bộ bằng Message Queue và Saga

Bên cạnh RESTful API, hệ thống sử dụng RabbitMQ kết hợp MassTransit để xử lý luồng nghiệp vụ bất đồng bộ. Trong phiên bản hiện tại, luồng đặt hàng không chỉ dừng ở việc phát sự kiện tạo đơn để trừ kho, mà đã được mở rộng thành Saga orchestration để điều phối các bước: tạo đơn, giữ tồn kho, xử lý thanh toán mô phỏng, hoàn tồn kho khi lỗi và cập nhật trạng thái đơn hàng.

Luồng bất đồng bộ chính được thiết kế như sau:

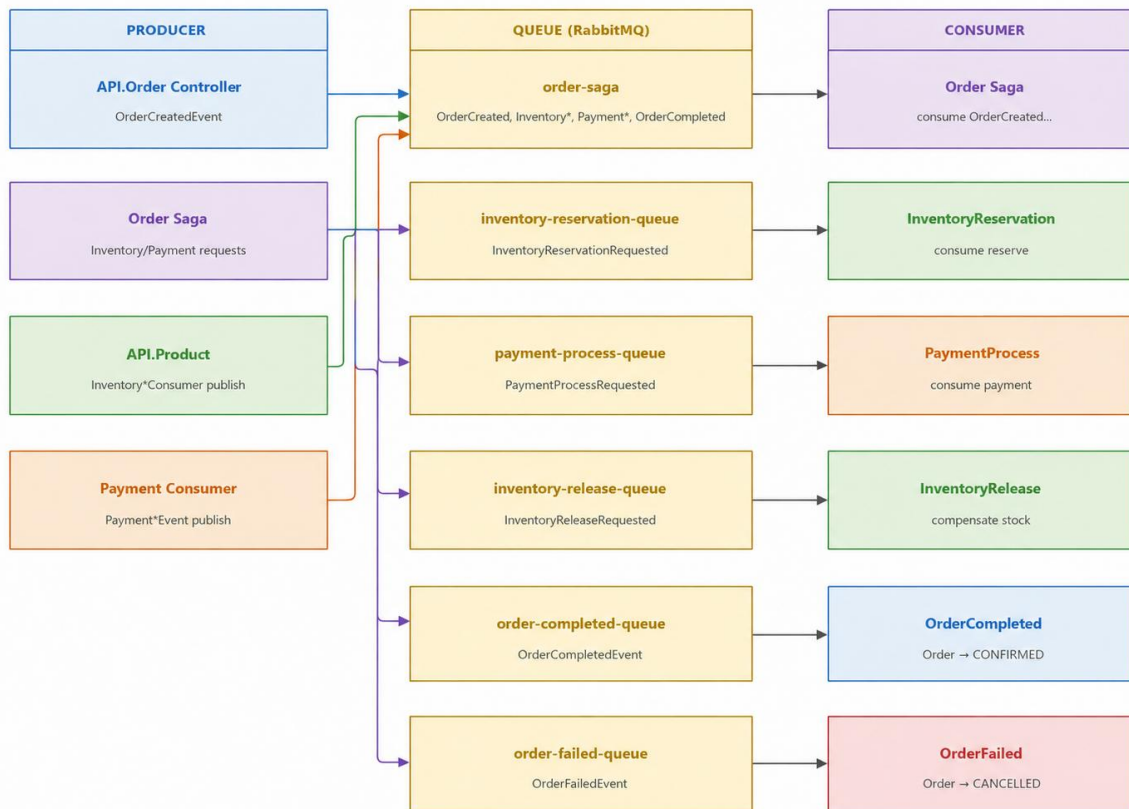
1. Order Service tạo đơn hàng, commit dữ liệu đơn hàng vào Order Database, sau đó publish OrderCreatedEvent.
2. Order Saga nhận OrderCreatedEvent và publish InventoryReservationRequested.
3. Product Service consume InventoryReservationRequested. Nếu đủ hàng, service trừ tồn kho, ghi inventory_transactions loại SALE và publish InventoryReservedEvent.

4. Nếu thiếu hàng hoặc có lỗi nghiệp vụ, Product Service publish InventoryReservationFailedEvent.
5. Khi giữ tồn kho thất bại, Saga publish OrderFailedEvent để OrderFailedConsumer cập nhật đơn hàng sang trạng thái CANCELLED.
6. Khi giữ tồn kho thành công, Saga publish PaymentProcessRequested.
7. PaymentProcessConsumer xử lý thanh toán mô phỏng. Nếu thành công, publish PaymentProcessedEvent; Saga tiếp tục publish OrderCompletedEvent và OrderCompletedConsumer cập nhật đơn hàng sang trạng thái CONFIRMED
8. Nếu thanh toán mô phỏng thất bại, PaymentProcessConsumer publish PaymentFailedEvent; Saga publish InventoryReleaseRequested.
9. Product Service consume InventoryReleaseRequested, hoàn tồn kho, ghi inventory_transactions loại RELEASE và publish InventoryReleasedEvent.
10. Saga nhận InventoryReleasedEvent và publish OrderFailedEvent để cập nhật đơn hàng sang trạng thái CANCELLED.



Hình 3.5. Sơ đồ tuần tự Saga điều phối luồng đặt hàng

Các queue chính đang sử dụng gồm:



Hình 3.6. Sơ đồ producer – hàng đợi (queue) – consumer trên RabbitMQ

Cách thiết kế này giúp hệ thống đạt nhất quán sau cùng có nhánh xử lý bù. Đơn hàng được ghi nhận trước, sau đó các bước giữ tồn kho và thanh toán mô phỏng được xử lý qua message. Nếu một bước quan trọng thất bại, Saga sẽ điều phối nhánh bù như hoàn tồn kho và hủy đơn hàng.

3.7. Thiết kế bảo mật

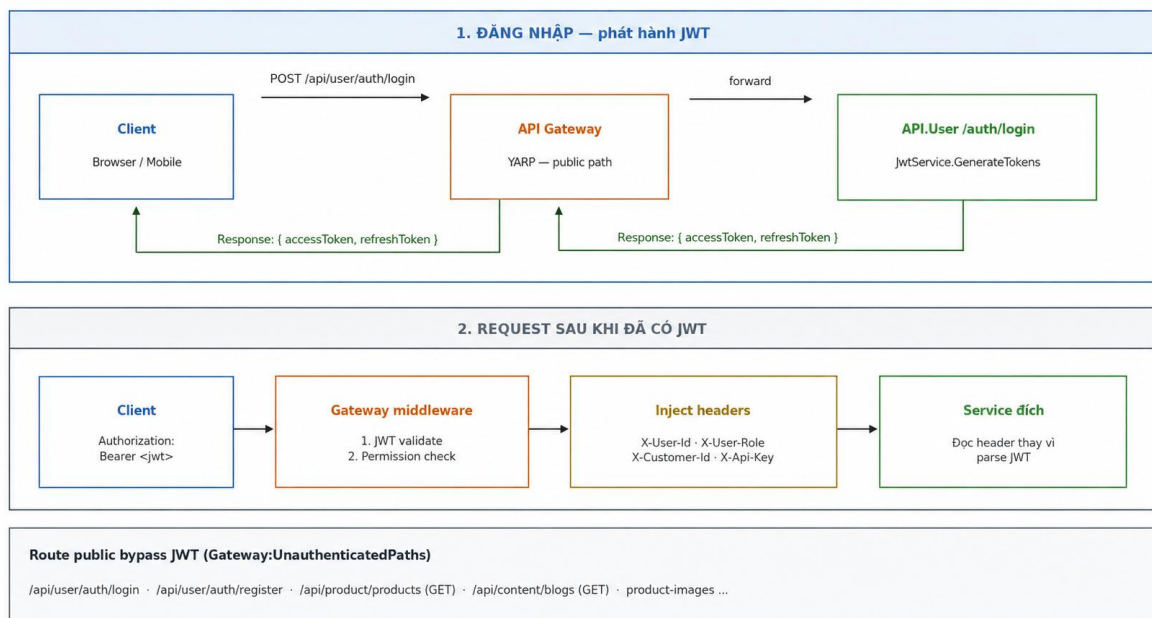
Hệ thống sử dụng cơ chế xác thực và phân quyền dựa trên JWT kết hợp với API Gateway. Khi người dùng đăng nhập thành công, hệ thống cấp token cho client. Các request đến route cần bảo vệ phải gửi kèm token này. Gateway kiểm tra token trước khi chuyển tiếp request đến service phía sau.

Sau khi xác thực thành công, Gateway bổ sung các thông tin ngữ cảnh người dùng vào header, ví dụ như mã người dùng, vai trò người dùng và mã khách hàng nếu có.

Các service phía sau sử dụng thông tin này để xử lý nghiệp vụ mà không cần tự phân tích lại token trong từng controller.

Ngoài JWT cho người dùng, hệ thống còn sử dụng API key nội bộ giữa Gateway và các service. Gateway tự động bổ sung X-API-Key vào các request chuyển tiếp. Các service phía sau kiểm tra API key ở những API có gắn cơ chế bảo vệ nội bộ. Một số endpoint public như đăng nhập, đăng ký, đọc sản phẩm/blog hoặc endpoint ảnh sản phẩm không gắn ApiKey để phục vụ truy cập công khai theo đúng phạm vi chức năng.

- Người dùng phải đăng nhập để sử dụng các chức năng cần xác thực.
- Route quản trị chỉ cho phép tài khoản có quyền phù hợp truy cập.
- Các API nghiệp vụ nội bộ có thể yêu cầu X-API-Key để hạn chế truy cập trực tiếp không đi qua Gateway.
- Thông tin người dùng được truyền xuống service thông qua header nội bộ.
- Các route public được khai báo rõ trong cấu hình UnauthenticatedPaths.



Hình 3.7. Luồng xác thực JWT và bổ sung header ngữ cảnh qua API Gateway

3.8. Thiết kế xử lý lỗi, logging và quan sát hệ thống

Trong hệ thống phân tán, lỗi có thể phát sinh ở nhiều vị trí khác nhau như frontend, Gateway, service nghiệp vụ, database, kết nối HTTP hoặc quá trình xử lý message. Vì

vậy, hệ thống cần có cơ chế xử lý lỗi, ghi log, tracing và metrics để hỗ trợ kiểm thử, debug và vận hành.

Hệ thống sử dụng cơ chế xử lý lỗi tập trung ở backend để chuẩn hóa phản hồi lỗi trả về client. Các service và Gateway sử dụng Serilog để ghi log; log được tập trung về Seq để dễ theo dõi. Ngoài ra, hệ thống tích hợp OpenTelemetry để thu thập tracing và metrics từ ASP.NET Core, HttpClient, SQL client và MassTransit. Trace được xem trên Jaeger, còn metrics được Prometheus scrape và hiển thị qua Grafana.

Về độ bền trong giao tiếp, các HTTP call liên service có resilience handler gồm retry, timeout và circuit breaker. Với giao tiếp bất đồng bộ, MassTransit consumer có retry policy để xử lý lỗi tạm thời khi consume message.

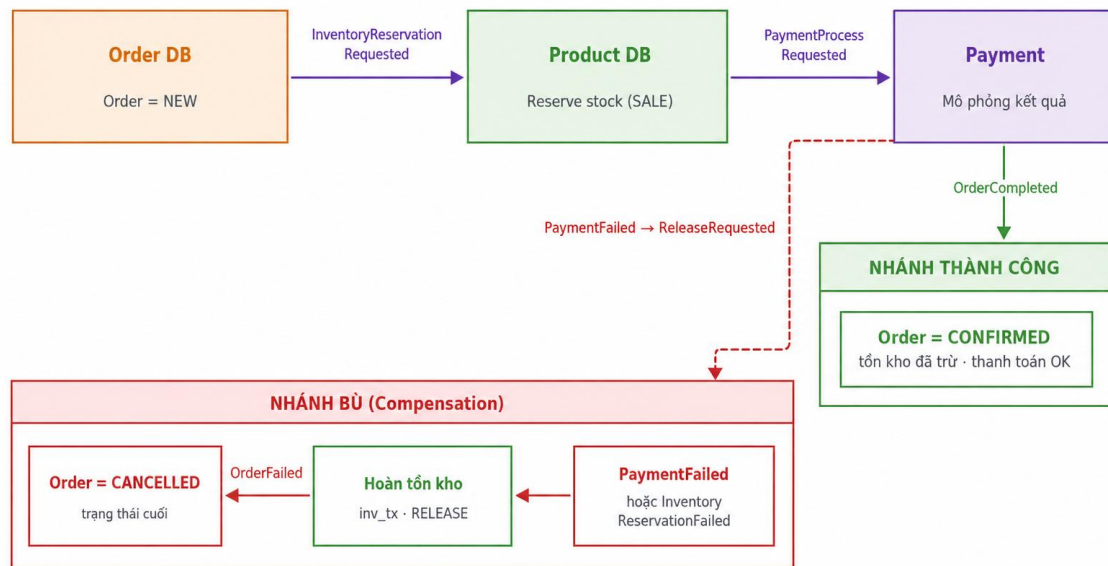
- Chuẩn hóa response lỗi trả về client.
- Ghi log request và lỗi phát sinh ở Gateway và từng service.
- Ghi log các sự kiện quan trọng trong Saga như tạo đơn, giữ tồn kho, thanh toán mô phỏng, hoàn tồn kho và cập nhật trạng thái đơn.
- Thu thập trace liên service bằng OpenTelemetry và Jaeger.
- Thu thập metrics bằng OpenTelemetry, Prometheus và Grafana.
- Áp dụng retry, timeout, circuit breaker cho HTTP call và retry policy cho MassTransit consumer.

3.9. Tính nhất quán dữ liệu trong hệ thống

Do hệ thống được thiết kế theo kiến trúc microservice và dữ liệu được tách theo từng service, không phải mọi nghiệp vụ đều có thể xử lý bằng một giao dịch duy nhất trên cùng một cơ sở dữ liệu. Luồng tạo đơn hàng, giữ tồn kho và thanh toán mô phỏng liên quan đến nhiều miền dữ liệu khác nhau, vì vậy hệ thống sử dụng Saga để điều phối thay cho giao dịch phân tán truyền thống.

Trong hệ thống FishShop, Order Service chịu trách nhiệm lưu đơn hàng vào Order Database, Product Service chịu trách nhiệm cập nhật tồn kho và lịch sử giao dịch kho trong Product Database. Sau khi đơn hàng được tạo, các bước tiếp theo được xử lý bằng event qua RabbitMQ. Nếu giữ tồn kho và thanh toán mô phỏng thành công, đơn hàng

được cập nhật hoàn tất khi luồng hoạt động thành công. Nếu một bước thất bại, Saga kích hoạt nhánh compensation và hủy đơn hàng.



Hình 3.8. Luồng nhất quán dữ liệu và xử lý bù (compensation) trong Saga

Cách thiết kế này hướng tới mô hình nhất quán sau cùng. Tại một thời điểm ngắn, trạng thái giữa đơn hàng và tồn kho có thể chưa đồng bộ hoàn toàn, nhưng sau khi các message được xử lý xong, hệ thống sẽ đạt trạng thái nghiệp vụ hợp lệ: đơn hoàn tất nếu toàn bộ bước thành công, hoặc đơn bị hủy và tồn kho được hoàn lại nếu có lỗi trong luồng xử lý.

CHƯƠNG 4. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG

4.1. Công nghệ sử dụng

Hệ thống được cài đặt theo kiến trúc microservice với nhiều thành phần độc lập. Các công nghệ được lựa chọn nhằm đáp ứng yêu cầu xây dựng RESTful API, giao tiếp bất đồng bộ bằng Message Queue, quản lý dữ liệu theo từng service, quan sát hệ thống và hỗ trợ triển khai trong môi trường demo.

Bảng 4.1. Công nghệ sử dụng trong hệ thống

Thành phần	Công nghệ sử dụng	Vai trò
Backend	.NET 8, ASP.NET Core Web API	Xây dựng các service nghiệp vụ và API
API Gateway	YARP Reverse Proxy	Định tuyến request, xác thực và chuyển tiếp đến service
ORM/Data Access	Entity Framework Core	Làm việc với cơ sở dữ liệu SQL Server
Database	SQL Server	Lưu trữ dữ liệu theo từng miền nghiệp vụ
Message Queue/Saga	RabbitMQ, MassTransit	Xử lý luồng bất đồng bộ và Saga orchestration
Logging	Serilog, Seq	Ghi log và theo dõi log tập trung
Tracing/Metrics	OpenTelemetry, Jaeger, Prometheus, Grafana	Thu thập trace, metrics và hiển thị dashboard giám sát
Resilience	HTTP resilience handler, MassTransit retry policy	Retry, timeout, circuit breaker và retry consumer

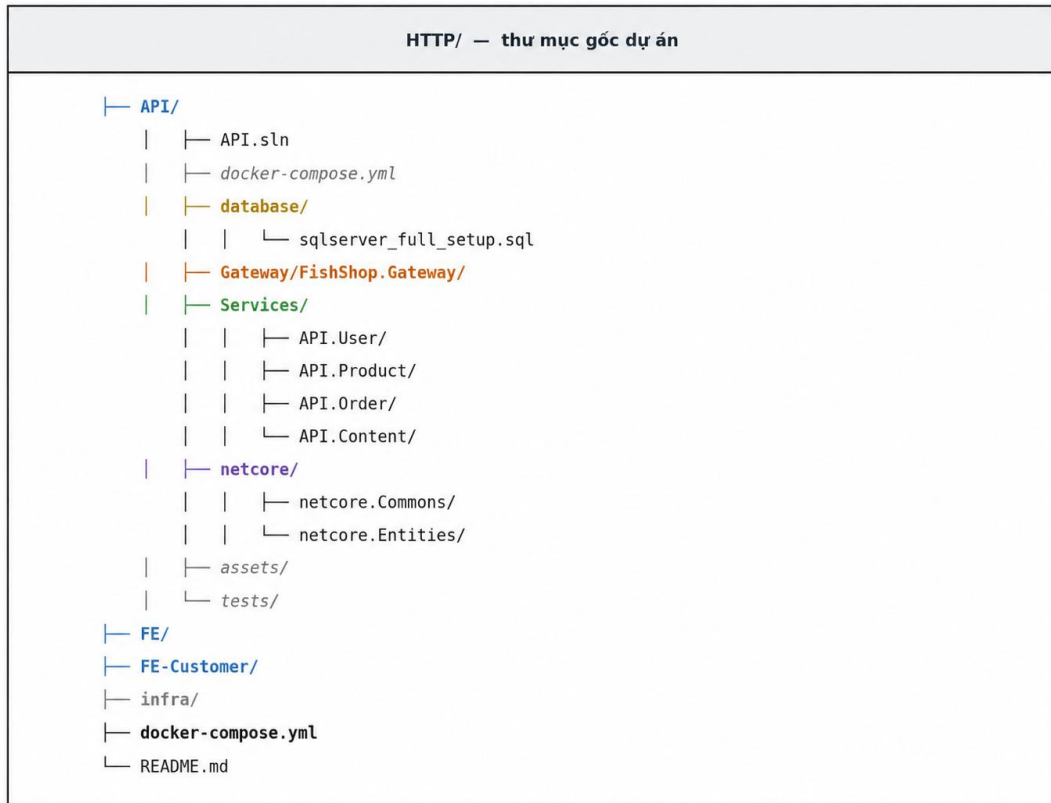
Frontend Admin	React, TypeScript, Vite	Giao diện quản trị
Frontend Customer	React, TypeScript, Vite	Giao diện khách hàng
Triển khai local	Docker Compose	Chạy nhiều service và hạ tầng phục vụ demo

Việc sử dụng Docker Compose giúp nhóm khởi chạy đồng thời các backend service, Gateway, RabbitMQ, Seq, Jaeger, Prometheus và Grafana trong cùng một môi trường. Frontend được chạy riêng bằng Vite, sau đó gọi API thông qua Gateway.

4.2. Cấu trúc mã nguồn

Mã nguồn hệ thống được tổ chức thành các thư mục riêng cho backend, frontend quản trị và frontend khách hàng. Backend được đặt trong thư mục API, bao gồm Gateway, các service nghiệp vụ, phần code dùng chung và script cơ sở dữ liệu. Frontend được tách thành hai ứng dụng riêng để phục vụ hai nhóm người dùng khác nhau.

Cấu trúc thư mục chính của project:



Hình 4.1. Cấu trúc thư mục chính của dự án

- **API/Gateway/FishShop.Gateway:** chứa API Gateway, cấu hình reverse proxy, middleware xác thực và bổ sung API key nội bộ.
- **API/Services/API.User:** xử lý xác thực, người dùng và khách hàng.
- **API/Services/API.Product:** xử lý sản phẩm, danh mục, ảnh, tồn kho, lịch sử kho và consumer tồn kho.
- **API/Services/API.Order:** xử lý giỏ hàng, đơn hàng, khuyến mãi, thanh toán logic, báo cáo và Saga orchestration.
- **API/Services/API.Content:** xử lý blog, danh mục blog và liên hệ khách hàng.
- **API/netcore/netcore.Commons:** chứa các thành phần dùng chung như response chuẩn, filter lỗi, attribute phân quyền, contract message, cấu hình observability/resilience dùng chung nếu có.
- **API/netcore/netcore.Entities:** chứa entity, DbContext, repository và Unit of Work.
- **FE và FE-Customer:** lần lượt là giao diện quản trị và giao diện khách hàng.

4.3. Cài đặt cơ sở dữ liệu

Hệ thống sử dụng SQL Server làm hệ quản trị cơ sở dữ liệu. Dữ liệu được tổ chức theo nguyên tắc tách cơ sở dữ liệu theo service. Mỗi service quản lý một database riêng, phù hợp với thiết kế microservice.

Bảng 4.2. Danh sách cơ sở dữ liệu theo service

Database	Service sử dụng	Nhóm dữ liệu
FishShop_User	User Service	roles, users, customer_profiles, customer_addresses
FishShop_Product	Product Service	categories, products, product_images, inventory, inventory_transactions
FishShop_Order	Order Service	shopping_carts, cart_items, orders, order_items, payments, promotions, promotion_products, promotion_usages
FishShop_Content	Content Service	blog_categories, blog_posts, contact_messages

Cơ sở dữ liệu được khởi tạo bằng script API/database/sqlserver_full_setup.sql. Script này thực hiện tạo lại các database, tạo bảng và seed dữ liệu mẫu phục vụ quá trình demo. Một số tài khoản mẫu được sử dụng để kiểm thử hệ thống gồm tài khoản quản trị viên và tài khoản khách hàng.

Bảng 4.3. Tài khoản mẫu phục vụ kiểm thử và demo

Vai trò	Username	Password
Quản trị viên (Admin)	admin	123456
Khách hàng	customer01	123456

Lệnh chạy script database có thể thực hiện như sau:

```
cd API\database
```

```
sqlcmd -S <sql-server> -U <username> -P <password> -i.\sqlserver_full_setup.sql
```

Trong đó, <sql-server>, <username> và <password> được thay bằng thông tin SQL Server trên máy triển khai.

4.4. Triển khai backend và hạ tầng

Backend của hệ thống gồm API Gateway và bốn service nghiệp vụ. Trong môi trường demo, các service backend được khởi chạy cùng với RabbitMQ, Seq, Jaeger, Prometheus và Grafana bằng Docker Compose. RabbitMQ phục vụ Message Queue/Saga, Seq phục vụ log tập trung, Jaeger phục vụ tracing, Prometheus thu thập metrics và Grafana hiển thị dashboard giám sát.

Lưu ý: SQL Server không được khởi tạo trong docker-compose.yml. SQL Server cần được cài đặt và chạy sẵn trên máy host (các service kết nối qua host.docker.internal), đồng thời cần chạy script sqlserver_full_setup.sql để tạo cơ sở dữ liệu và nạp dữ liệu mẫu trước khi khởi động backend.

Bảng 4.4. Danh sách container backend và hạ tầng triển khai

Container	Vai trò
fishshop-gateway	API Gateway, điểm vào chung của backend
api-user	Service người dùng và xác thực
api-product	Service sản phẩm, danh mục, tồn kho

api-order	Service giỏ hàng, đơn hàng, khuyến mãi, thanh toán logic và Saga
api-content	Service blog và liên hệ
rabbitmq-service	Message Queue
seq-service	Theo dõi log tập trung
jaeger-service	Theo dõi distributed trace
prometheus-service	Scrape và lưu metrics
grafana-service	Dashboard giám sát metrics

Lệnh khởi chạy backend và hạ tầng:

```
docker compose up -d --build
```

Sau khi chạy, có thể kiểm tra trạng thái các container bằng lệnh:

```
docker compose ps
```

Bảng 4.5. Địa chỉ truy cập các thành phần sau triển khai

Thành phần	Địa chỉ
API Gateway	http://localhost:5000
RabbitMQ Management UI	http://localhost:15672
Seq UI	http://localhost:5341
Jaeger UI	http://localhost:16686
Prometheus	http://localhost:9090
Grafana	http://localhost:3002

Trong Docker Compose, Gateway được public ra cổng 5000, còn các service backend chạy trong network nội bộ Docker. Khi frontend gọi API, request được gửi đến Gateway, sau đó Gateway định tuyến đến service tương ứng.

4.5. Cấu hình backend quan trọng

Phần này trình bày những nhóm cấu hình cốt lõi quyết định cách backend vận hành theo kiến trúc microservice: định tuyến qua API Gateway, kết nối cơ sở dữ liệu riêng cho từng service, giao tiếp bất đồng bộ qua RabbitMQ và các biến môi trường phục vụ quan sát hệ thống. Thay vì liệt kê toàn bộ tệp cấu hình, nội dung dưới đây tập trung giải thích vai trò của từng nhóm và nêu các giá trị quan trọng được trích từ mã nguồn thực tế, làm cơ sở để giải thích cơ chế vận hành của hệ thống.

4.5.1. Định tuyến qua API Gateway (YARP)

API Gateway sử dụng thư viện YARP (Yet Another Reverse Proxy) làm điểm vào duy nhất của backend. Thay vì để frontend gọi trực tiếp tới từng service, Gateway tiếp nhận toàn bộ request rồi chuyển tiếp tới service phù hợp dựa trên tiền tố đường dẫn. Mỗi nhóm đường dẫn dạng `/api/{service}/...` được khai báo thành một route trong `appsettings.json` và ánh xạ tới một cluster trở về service tương ứng.

Cụ thể, request bắt đầu bằng `/api/user` được định tuyến tới User Service, `/api/product` tới Product Service, `/api/order` tới Order Service và `/api/content` tới Content Service; phần đường dẫn phía sau được giữ nguyên (catch-all) khi chuyển tiếp. Khi backend chạy trực tiếp bằng .NET, bốn cluster này lần lượt trở tới các cổng cục bộ 5001, 5002, 5003 và 5005.

Khi triển khai bằng Docker Compose, địa chỉ đích của mỗi cluster được ghi đè qua biến môi trường để trở tới tên service nội bộ trong mạng Docker, ví dụ `http://api-user:8080/` hoặc `http://api-product:8080/`. Nhờ vậy, frontend chỉ cần biết một địa chỉ Gateway duy nhất, còn số lượng và vị trí thật của các service bên trong được che giấu hoàn toàn.

4.5.2. Chuỗi kết nối cơ sở dữ liệu

Theo nguyên tắc database-per-service, mỗi service sở hữu một cơ sở dữ liệu riêng và tự cấu hình chuỗi kết nối tới database của mình trong `appsettings.json`, không dùng chung kết nối với service khác. Tương ứng, `API.User` kết nối tới `FishShop_User`, `API.Product` tới `FishShop_Product`, `API.Order` tới `FishShop_Order` và `API.Content` tới

FishShop_Content. Các chuỗi kết nối tuân theo cùng một mẫu, chỉ khác nhau ở tên database:

```
Server=host.docker.internal;Database=FishShop_<Service>;User  
Id=sa;Password=viet123;TrustServerCertificate=True;
```

Khi chạy backend trực tiếp bằng .NET (không qua Docker), có thể đổi host.docker.internal thành localhost.

4.5.3. Cấu hình RabbitMQ và MassTransit

Với giao tiếp bất đồng bộ, hệ thống sử dụng MassTransit chạy trên nền RabbitMQ. Host RabbitMQ được cấu hình qua biến môi trường RabbitMQ__Host (mặc định là rabbitmq trong môi trường Docker hoặc localhost khi chạy trực tiếp) với tài khoản mặc định guest/guest. Để tách biệt trách nhiệm xử lý, mỗi bước trong luồng đặt hàng được gắn với một hàng đợi riêng.

Cụ thể, hàng đợi order-saga do Order Saga (OrderStateMachine) đảm nhận việc điều phối toàn bộ luồng; inventory-reservation-queue và inventory-release-queue thuộc Product Service, lần lượt phụ trách giữ/trừ tồn kho và hoàn tồn kho; còn payment-process-queue, order-completed-queue và order-failed-queue thuộc Order Service, lần lượt xử lý thanh toán mô phỏng, cập nhật đơn sang trạng thái CONFIRMED và cập nhật đơn sang trạng thái CANCELLED.

Như vậy, Order Service đăng ký Saga cùng các consumer xử lý thanh toán, hoàn tất và hủy đơn, trong khi Product Service đăng ký các consumer giữ kho và hoàn kho. Mỗi consumer được cấu hình retry policy để tự động thử lại khi gặp lỗi tạm thời trong lúc tiêu thụ message.

4.5.4. Biến môi trường và quan sát hệ thống

Cuối cùng, mỗi service nhận thêm một số biến môi trường phục vụ quan sát hệ thống. Biến Seq__ServerUrl trỏ tới Seq để gom log tập trung, còn Otlp__Endpoint trỏ tới Jaeger qua cổng OTLP gRPC 4317 để gửi dữ liệu trace. Riêng Order Service có thêm ProductServiceUrl và UserServiceUrl là địa chỉ dùng để gọi REST liên service khi xử lý đơn hàng. Toàn bộ thành phần cùng nằm trong mạng nội bộ Docker fishshop_network; trong đó chỉ Gateway được public ra cổng 5000, còn các service

ng nghiệp vụ không expose trực tiếp ra ngoài nhằm bảo đảm mọi truy cập đều phải đi qua Gateway.

4.6. Triển khai frontend

Hệ thống có hai ứng dụng frontend riêng biệt: FE là giao diện dành cho quản trị viên/nhân viên và FE-Customer là giao diện dành cho khách hàng. Cả hai frontend đều gọi backend thông qua API Gateway. Khi chạy ở môi trường phát triển, frontend sử dụng proxy /api trở về <http://localhost:5000>, giúp các request từ giao diện được chuyển đến Gateway.

4.6.1. Chạy frontend quản trị

Frontend quản trị được đặt trong thư mục FE. Ứng dụng này phục vụ các chức năng như dashboard, quản lý sản phẩm, danh mục, tồn kho, đơn hàng, khách hàng, khuyến mãi, blog và báo cáo.

```
cd FE
```

```
npm install
```

```
npm run dev
```

Địa chỉ truy cập: <http://localhost:3000>

4.6.2. Chạy frontend khách hàng

Frontend khách hàng được đặt trong thư mục FE-Customer. Ứng dụng này phục vụ các chức năng như xem sản phẩm, tìm kiếm sản phẩm, xem blog, đăng nhập, giỏ hàng, checkout, đơn hàng của tôi và hồ sơ cá nhân.

```
cd FE-Customer
```

```
npm install
```

```
npm run dev
```

Địa chỉ truy cập: <http://localhost:3001>

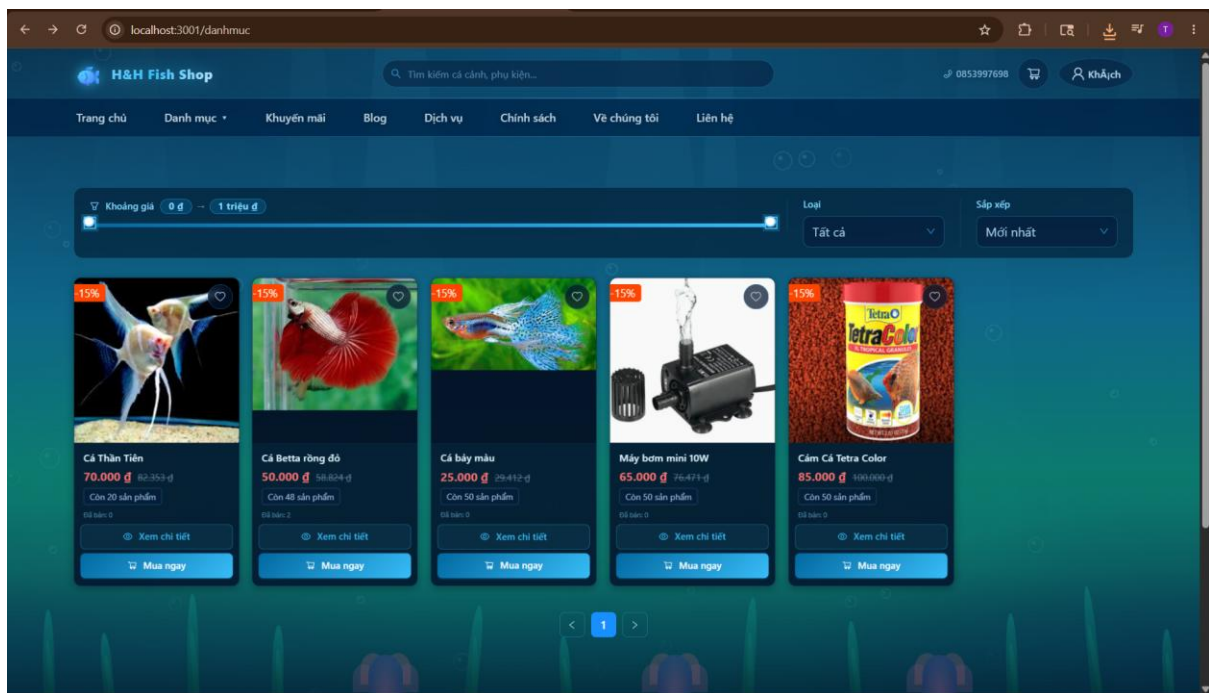
4.7. Giao diện hệ thống

Sau khi triển khai backend, hạ tầng và frontend, hệ thống có thể được sử dụng thông qua hai giao diện chính: giao diện khách hàng và giao diện quản trị.

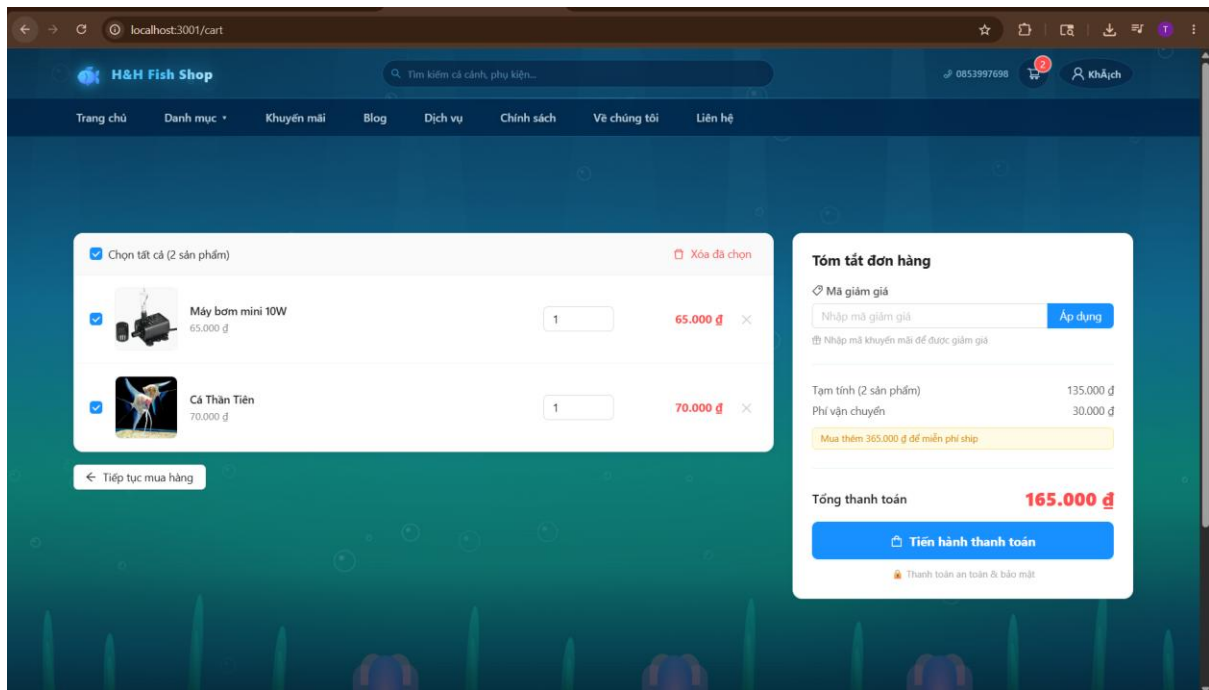
4.7.1. Giao diện khách hàng

Giao diện khách hàng hỗ trợ các thao tác mua hàng cơ bản. Người dùng có thể xem danh sách sản phẩm, xem chi tiết sản phẩm, thêm sản phẩm vào giỏ hàng, đặt hàng và theo dõi đơn hàng.

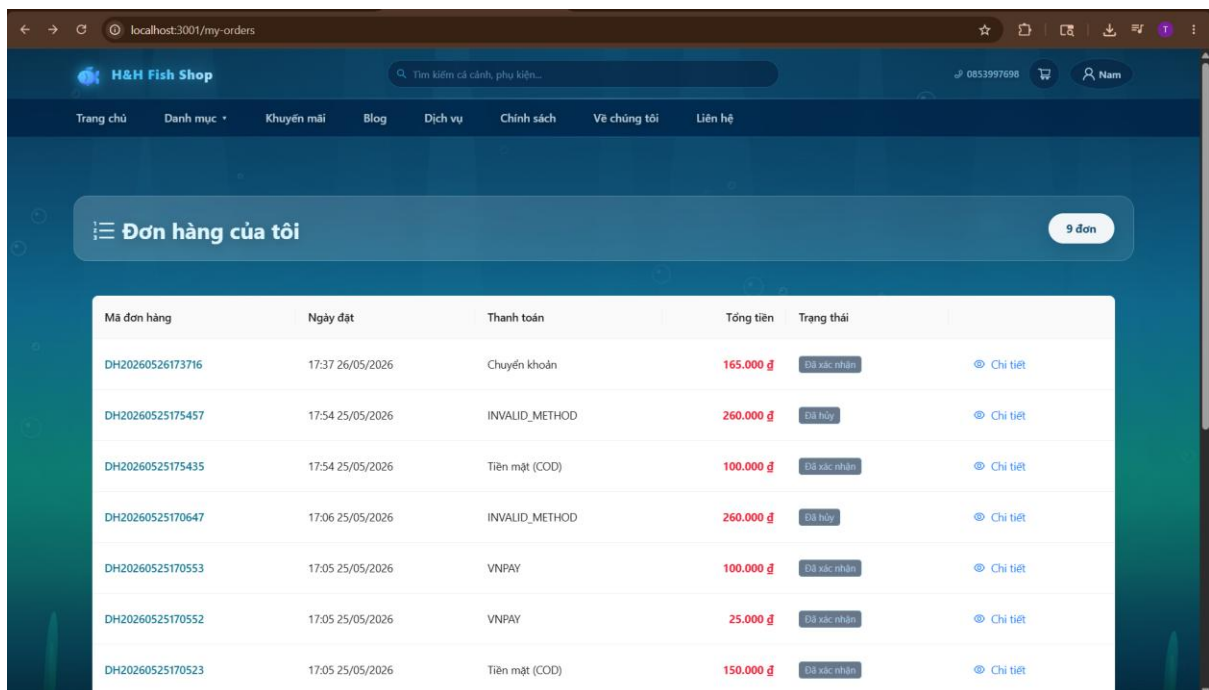
- Trang chủ hoặc danh sách sản phẩm.
- Trang chi tiết sản phẩm.
- Trang giỏ hàng.
- Trang checkout/đặt hàng.
- Trang đơn hàng của tôi.
- Trang blog hoặc liên hệ.



Hình 4.2. Trang danh sách sản phẩm phía khách hàng



Hình 4.3. Trang giỏ hàng và thanh toán (checkout)

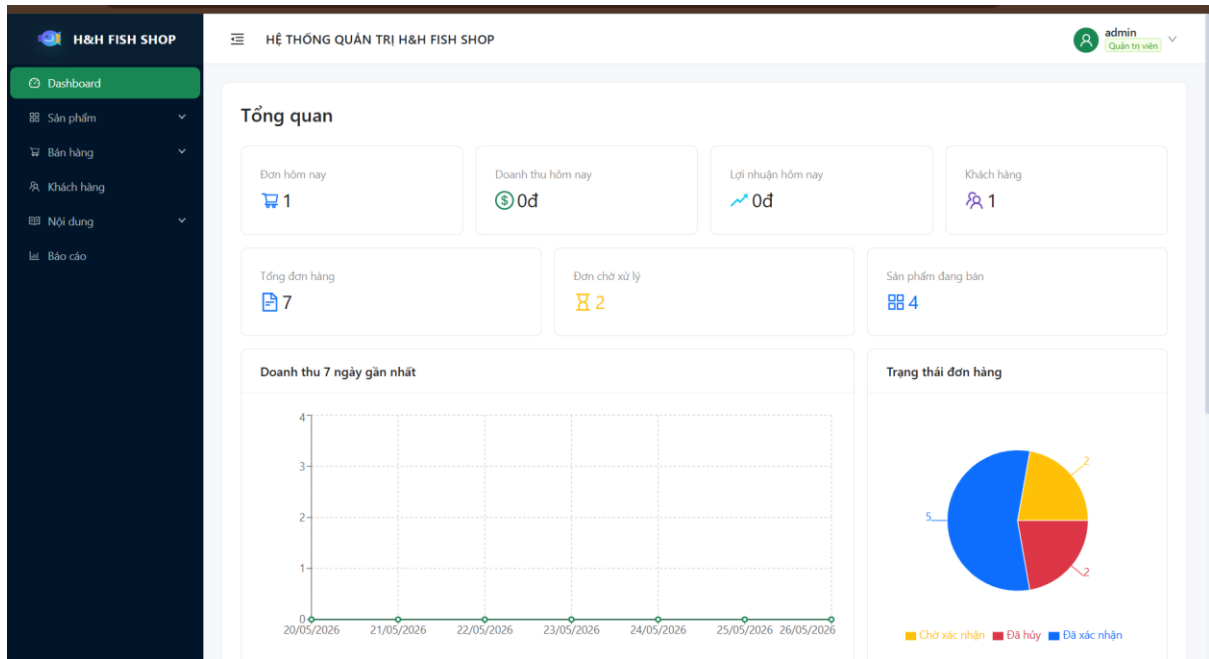


Hình 4.4. Trang theo dõi đơn hàng của khách hàng

4.7.2. Giao diện quản trị

Giao diện quản trị hỗ trợ nhân viên hoặc quản trị viên vận hành hệ thống. Người quản trị có thể quản lý sản phẩm, danh mục, tồn kho, đơn hàng, khách hàng, blog, liên hệ và xem các thông tin thống kê.

- Dashboard quản trị.
- Quản lý sản phẩm.
- Quản lý đơn hàng.
- Quản lý khách hàng.
- Quản lý blog/liên hệ.
- Báo cáo hoặc thống kê nếu có.



Hình 4.5. Trang tổng quan (dashboard) quản trị

The product management interface, titled "SẢN PHẨM", allows for the management of the shop's inventory. It includes a search bar, a filter dropdown, and a "+ Thêm sản phẩm" button. The product list is as follows:

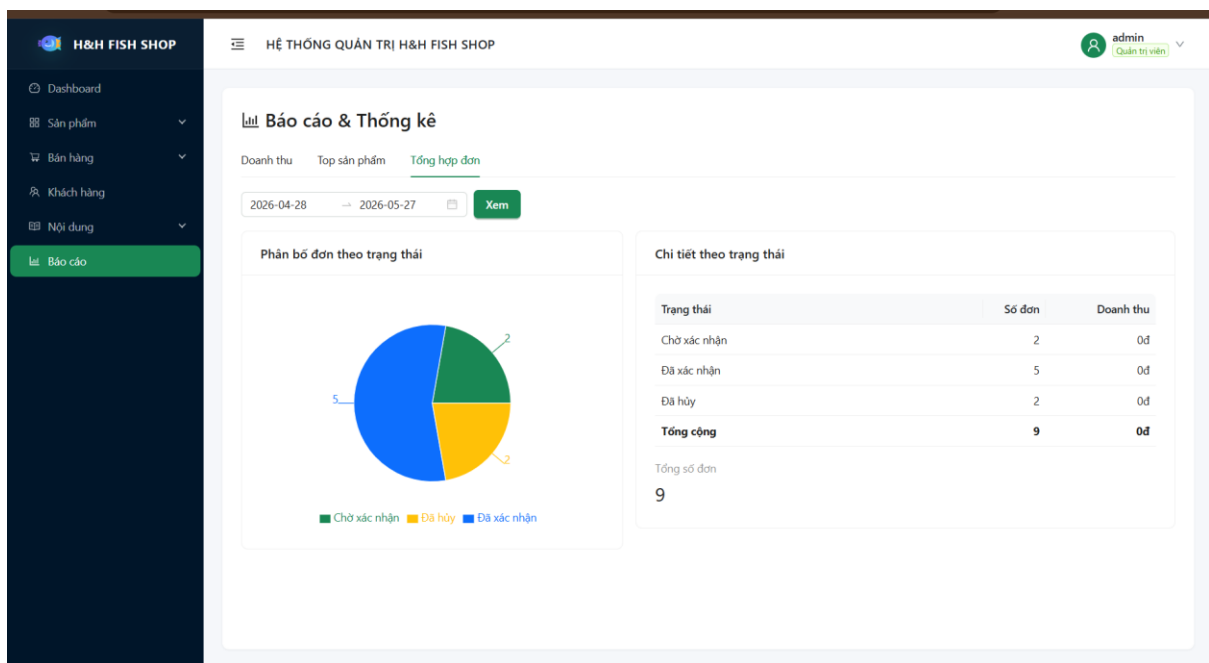
Mã SP	Ảnh	Tên sản phẩm	Danh mục	Giá bán	Tồn kho	Đã bán	Hành động
SP202605 26173435		Cá Thần Tiên	Cá Cảnh	70.000đ	19	1	
P1		Cá Betta rồng đỏ	Cá Cảnh	50.000đ	48	2	
P2		Cá bảy màu	Cá Cảnh	25.000đ	50	0	
P3		Máy bơm mini 10W	Phụ kiện	65.000đ	49	1	
P4		Cám Cá Tetra Color	Thức ăn	85.000đ	50	0	

At the bottom, it shows "Tổng 5 sản phẩm" and a pagination control for "1" of "10 / trang".

Hình 4.6. Màn hình quản lý sản phẩm

Mã đơn	Khách hàng	Tổng tiền	Nguồn	Thanh toán	Trạng thái	Ngày tạo	Chi tiết
DH28268526173716	Nguyễn Văn Nam 0901234569	165.000đ	Online	Chờ thanh toán	Đã xác nhận	26/05/2026	Xem
DH28268525175457	T 0900000000	260.000đ	Online	Chờ thanh toán	Đã hủy	25/05/2026	Xem
DH28268525175435	T 0900000000	100.000đ	Online	Chờ thanh toán	Đã xác nhận	25/05/2026	Xem
DH28268525178647	T 0900000000	260.000đ	Online	Chờ thanh toán	Đã hủy	25/05/2026	Xem
DH28268525178553	T 0900000000	100.000đ	Online	Chờ thanh toán	Đã xác nhận	25/05/2026	Xem
DH28268525178552	T 0900000000	25.000đ	Online	Chờ thanh toán	Đã xác nhận	25/05/2026	Xem

Hình 4.7. Màn hình quản lý đơn hàng



Hình 4.8. Màn hình quản lý blog và liên hệ

4.8. Kiểm tra nhanh sau triển khai

Sau khi chạy hệ thống, nhóm thực hiện kiểm tra nhanh để đảm bảo Gateway và các service hoạt động đúng. Các endpoint health-check được gọi thông qua Gateway:

GET <http://localhost:5000/api/user/auth/health>

GET <http://localhost:5000/api/product/products/health>

GET http://localhost:5000/api/order/orders/health

GET http://localhost:5000/api/content/blogs/health

GET New Request

http://localhost:5000/api/user/auth/health

200 • 14 ms • 303 B

No tests found

GET New Request

http://localhost:5000/api/product/products/health

200 • 18 ms • 306 B

No tests found

GET New Request

http://localhost:5000/api/order/orders/health

200 • 23 ms • 304 B

No tests found

GET New Request

http://localhost:5000/api/content/blogs/health

200 • 72 ms • 306 B

No tests found

Hình 4.9. Kết quả gọi health-check các service qua API Gateway

Ngoài ra, nhóm có thể kiểm tra dữ liệu sản phẩm public bằng endpoint:

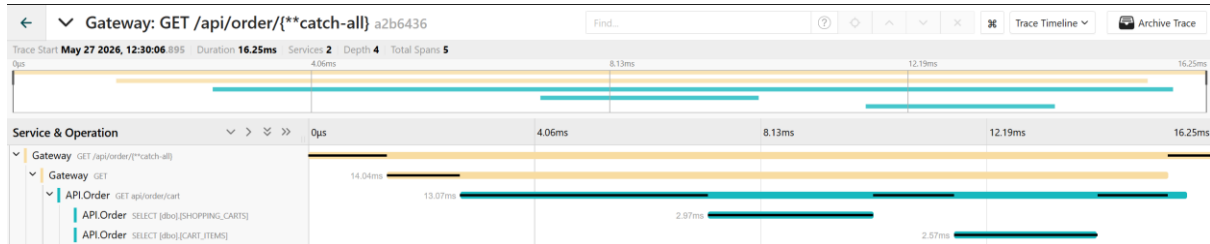
GET http://localhost:5000/api/product/products?page=1&pageSize=2

The screenshot shows a REST client interface with a GET request to `http://localhost:5000/api/product/products?page=1&pageSize=2`. The response is a 200 OK status with a 17 ms response time and 1.04 KB of data. The response body is in JSON format, showing a list of two products. The first product is 'Cá Betta Rồng Đỏ' and the second is 'Cá Bảy Màu Thái'. Both products are in stock and have a status of 'true'. The response also includes pagination information: total count is 4, page is 1, page size is 2, total pages is 2, and there is a next page.

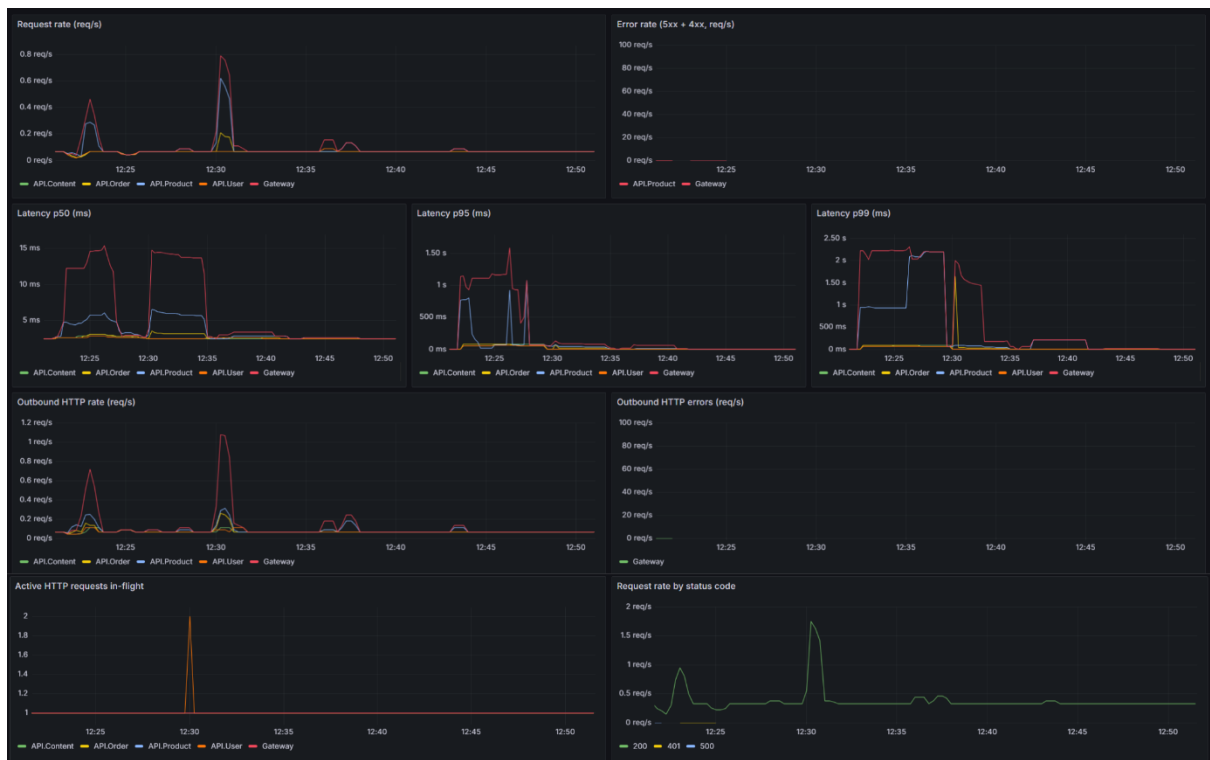
```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": 1,
        "productCode": "P1",
        "name": "Cá Betta Rồng Đỏ",
        "slug": "ca-betta-rong-do",
        "description": "Cá Betta Rồng Đỏ cực đẹp",
        "imageUrl": "betta-red.jpg",
        "salePrice": 50000.00,
        "stockQuantity": 0,
        "soldQuantity": 0,
        "status": true,
        "isFeatured": false,
        "categoryId": 1,
        "categoryName": "Cá cảnh",
        "createdAt": "2026-05-27T05:28:36.8907557"
      },
      {
        "id": 2,
        "productCode": "P2",
        "name": "Cá Bảy Màu Thái",
        "slug": "ca-bay-mau-thai",
        "description": "Cá Bảy Màu Thái thuần chủng",
        "imageUrl": "guppy-thai.jpg",
        "salePrice": 25000.00,
        "stockQuantity": 0,
        "soldQuantity": 0,
        "status": true,
        "isFeatured": false,
        "categoryId": 1,
        "categoryName": "Cá cảnh",
        "createdAt": "2026-05-27T05:28:36.8907557"
      }
    ],
    "totalCount": 4,
    "page": 1,
    "pageSize": 2,
    "totalPages": 2,
    "hasPreviousPage": false,
    "hasNextPage": true
  },
  "message": "Lấy danh sách sản phẩm thành công"
}
```

Hình 4.10. Kết quả gọi API danh sách sản phẩm qua API Gateway

Sau khi các endpoint trả về trạng thái thành công, có thể tiếp tục kiểm thử luồng nghiệp vụ như đăng nhập, thêm giỏ hàng, tạo đơn hàng, theo dõi Saga, kiểm tra queue RabbitMQ, xem trace trên Jaeger và kiểm tra metrics trên Grafana/Prometheus.



Hình 4.11. Trace trên Jaeger sau khi tạo đơn hàng



Hình 4.12. Dashboard Grafana sau khi tạo đơn hàng

CHƯƠNG 5. KIỂM THỬ HỆ THỐNG

5.1. Mục tiêu kiểm thử

Quá trình kiểm thử nhằm xác minh hệ thống FishShop hoạt động đúng theo yêu cầu nghiệp vụ và thể hiện được đặc trưng của kiến trúc microservice. Cụ thể, kiểm thử tập trung vào các mục tiêu sau:

- Xác nhận từng service hoạt động độc lập và phản hồi đúng qua API Gateway.
- Kiểm tra luồng nghiệp vụ chính từ phía khách hàng: đăng ký, đăng nhập, xem sản phẩm, thêm giỏ hàng và tạo đơn hàng.
- Kiểm tra luồng Saga đặt hàng ở cả trường hợp thành công (happy path) và các trường hợp thất bại có xử lý bù (compensation).
- Kiểm tra cơ chế bảo mật: xác thực JWT, bảo vệ route và phân quyền cơ bản.
- Quan sát log, message và trace giữa các service để xác nhận hệ thống phối hợp đúng và có khả năng theo dõi (observability).

5.2. Môi trường kiểm thử

Hệ thống được kiểm thử trong môi trường phát triển và demo cục bộ. Backend và hạ tầng quan sát được khởi chạy bằng Docker Compose, SQL Server chạy trên máy host, hai frontend chạy bằng Vite. Các thành phần và địa chỉ truy cập chính như sau:

Bảng 5.1. Môi trường và công cụ kiểm thử

Thành phần	Địa chỉ / Ghi chú
API Gateway	http://localhost:5000
Frontend khách hàng / quản trị	http://localhost:3001 và http://localhost:3000
RabbitMQ Management	http://localhost:15672 (guest/guest)
Seq (log tập trung)	http://localhost:5341
Jaeger (tracing)	http://localhost:16686

Prometheus / Grafana (metrics)	http://localhost:9090 và http://localhost:3002
SQL Server	Chạy trên máy host, nạp dữ liệu bằng sqlserver_full_setup.sql

5.3. Phương pháp và công cụ kiểm thử

Trong phạm vi đồ án, hệ thống được kiểm thử thủ công theo kịch bản, chưa xây dựng bộ kiểm thử tự động (unit test, integration test). Việc kiểm thử được thực hiện bằng các công cụ sau:

- Tập API/tests/interservice-order-flow.http: kịch bản kiểm thử luồng liên service qua Gateway, chạy bằng REST Client của VS Code hoặc nhập vào Postman.
- Postman / REST Client: gọi trực tiếp các endpoint để kiểm tra phản hồi và mã trạng thái HTTP.
- RabbitMQ Management: quan sát các hàng đợi và message của Saga.
- Seq, Jaeger, Grafana: theo dõi log, trace và metrics để xác nhận luồng xử lý giữa các service.

Tập interservice-order-flow.http kiểm tra tuần tự: health-check bốn service, đăng nhập khách hàng customer01, đọc sản phẩm qua API.Product, xóa giỏ, thêm sản phẩm vào giỏ, tạo đơn hàng qua API.Order, xem chi tiết đơn, và đọc lại sản phẩm để xác nhận Saga đã gửi message reserve tồn kho sang API.Product qua RabbitMQ.

5.4. Các kịch bản kiểm thử

Các kịch bản kiểm thử chính và kết quả được tổng hợp trong bảng dưới đây. Kết quả thực tế được ghi nhận khi chạy hệ thống trong môi trường demo nêu trên.

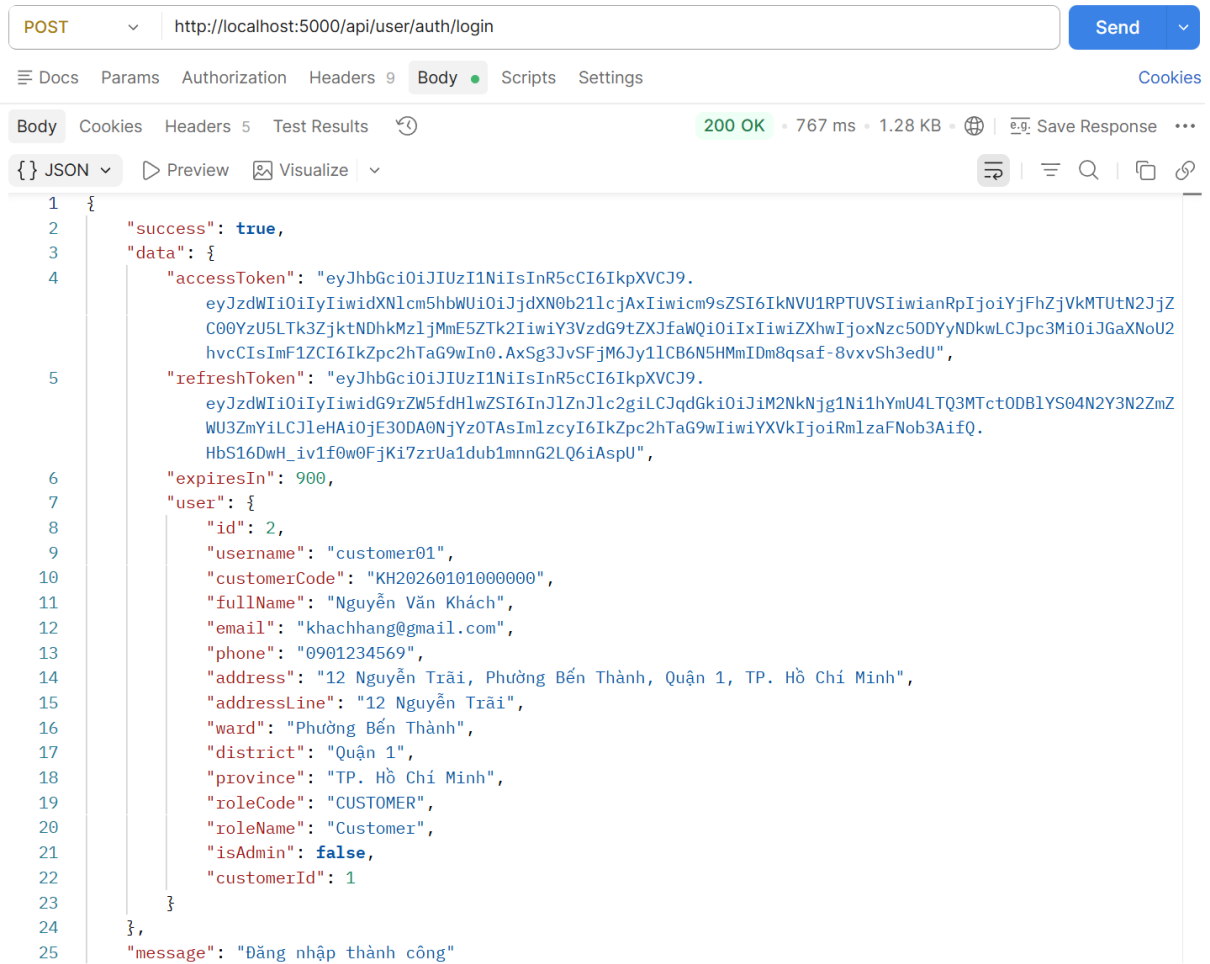
Bảng 5.2. Tổng hợp các kịch bản kiểm thử chính

Mã test	Chức năng	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-01	Đăng ký tài khoản	POST /api/user/auth/register với	Tạo tài khoản thành công, trả về	Tài khoản được tạo, đăng nhập được	Đạt

		username, email, mật khẩu mới	thông tin người dùng		
TC-02	Đăng nhập	POST /api/user/auth/login {customer01 / 123456}	Trả về accessToken (JWT) và refreshToken	Nhận đủ token, gọi được các API cần xác thực	Đạt
TC-03	Định tuyến qua Gate way	GET health-check của 4 service qua /api/{service}/...	Gateway chuyển đúng tới từng service, trả mã 200	Bốn endpoint health đều phản hồi thành công	Đạt
TC-04	Xem sản phẩm / danh mục	GET /api/product/products?page=1&pageSize=3 (route public)	Trả danh sách sản phẩm phân trang, không cần token	Trả đúng danh sách sản phẩm	Đạt
TC-05	Thêm vào giỏ hàng	POST /api/order/carts {productId:1, quantity:1} kèm JWT	Sản phẩm được thêm; GET /api/order/carts hiển thị item	Giỏ hàng cập nhật đúng số lượng và giá	Đạt
TC-06	Tạo đơn hàng	POST /api/order/orders {paymentMethod: COD} từ giỏ hàng	Tạo đơn, trạng thái ban đầu NEW, trả về orderCode	Đơn được tạo kèm orderCode	Đạt
TC-07	Saga - happy path	Đặt hàng sản phẩm còn đủ tồn kho	OrderCreated → giữ kho (SALE) → thanh toán thành công → đơn CONFIRMED; tồn kho giảm	Đơn chuyển CONFIRMED, tồn kho giảm, có giao dịch kho SALE	Đạt

TC-08	Saga - hết hàng	Đặt hàng với số lượng lớn hơn tồn kho	Product publish InventoryReservationFailed → Saga → đơn CANCELLED	Đơn chuyển CANCELLED, tồn kho không thay đổi	Đạt
TC-09	Saga - thanh toán lỗi	Mô phỏng thanh toán thất bại sau khi đã giữ kho	PaymentFailed → InventoryReleaseRequested → hoàn kho (RELEASE) → đơn CANCELLED	Đơn CANCELLED, tồn kho được hoàn lại đúng số lượng	Đạt
TC-10	Bảo vệ route / phân quyền	Gọi API cần đăng nhập (vd POST /api/order/orders) không kèm JWT	Gateway từ chối với mã 401 Unauthorized	Request bị chặn với 401	Đạt
TC-11	Race condition tồn kho	Nhiều request đặt cùng một sản phẩm tồn thấp gần như đồng thời	Không bán vượt quá số lượng tồn kho	Hệ thống kiểm tra tồn kho trước khi trừ và chống xử lý lặp theo orderCode; tuy nhiên chưa có khóa lạc quan (optimistic concurrency) nên vẫn có thể bán vượt khi tải cao	Hạn chế

Một số hình ảnh minh họa kết quả kiểm thử:



Hình 5.1. Kết quả đăng nhập trên Postman

POST

http://localhost:5000/api/order/orders

Send

Docs Params Authorization Headers 9 Body Scripts Settings

Cookies

Body Cookies Headers 5 Test Results

200 OK • 478 ms • 913 B • Save Response

JSON Preview Visualize

```

1 {
2   "success": true,
3   "data": {
4     "id": 1,
5     "orderCode": "DH20260527061639",
6     "customerId": 2,
7     "customerName": "Nguyễn Văn Khách",
8     "customerPhone": "0901234569",
9     "orderStatus": "PENDING",
10    "paymentMethod": "COD",
11    "paymentStatus": "PENDING",
12    "subtotal": 50000.00,
13    "discountAmount": 0.00,
14    "shippingFee": 0.00,
15    "totalAmount": 50000.00,
16    "shippingAddress": "12 Nguyễn Trãi, Phường Bến Thành, Quận 1, TP. Hồ Chí Minh",
17    "notes": "Demo báo cáo",
18    "source": "ONLINE",
19    "createdAt": "2026-05-27T06:16:39.0177343Z",
20    "updatedAt": "2026-05-27T06:16:39.0177432Z",
21    "items": [
22      {
23        "id": 1,
24        "productId": 1,
25        "productName": "Cá Betta Rồng Đỏ",
26        "imageUrl": "betta-red.jpg",
27        "quantity": 1,
28        "unitPrice": 50000.00,
29        "subTotal": 50000.00
30      }
31    ]
32  },
33   "message": "Đặt hàng thành công"
34 }

```

GET

http://localhost:5000/api/order/orders/DH20260527061639

Send

Docs Params Authorization Headers 7 Body Scripts Settings

Cookies

Body Cookies Headers 5 Test Results

200 OK • 16 ms • 875 B • Save Response

JSON Preview Visualize

```

2   "success": true,
3   "data": {
4     "id": 1,
5     "orderCode": "DH20260527061639",
6     "customerId": 2,
7     "customerName": "Nguyễn Văn Khách",
8     "customerPhone": "0901234569",
9     "orderStatus": "CONFIRMED",
10    "paymentMethod": "COD",
11    "paymentStatus": "PENDING",
12    "subtotal": 50000.00,
13    "discountAmount": 0.00,
14    "shippingFee": 0.00,
15    "totalAmount": 50000.00,
16    "shippingAddress": "12 Nguyễn Trãi, Phường Bến Thành, Quận 1, TP. Hồ Chí Minh",
17    "notes": "Demo báo cáo",
18    "source": "ONLINE",
19    "createdAt": "2026-05-27T06:16:39.0177343",
20    "updatedAt": "2026-05-27T06:16:40.0025335",
21    "items": [
22      {
23        "id": 1,
24        "productId": 1,
25        "productName": "Cá Betta Rồng Đỏ",
26        "imageUrl": "betta-red.jpg",
27        "quantity": 1,
28        "unitPrice": 50000.00,
29        "subTotal": 50000.00
30      }
31    ]
32  }

```

Hình 5.2. Kết quả tạo đơn hàng thành công

Queues

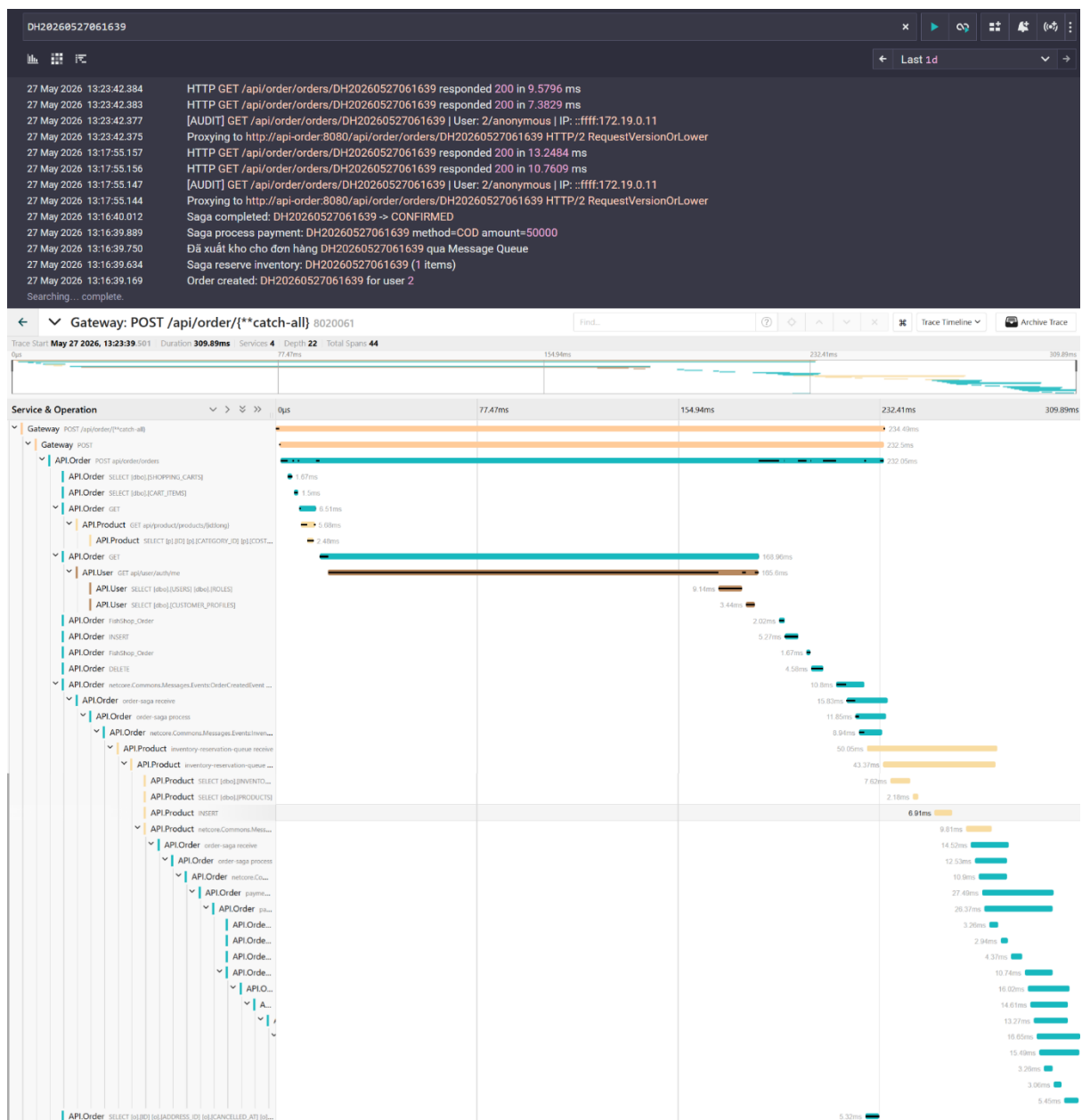
▼ All queues (6)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Overview					Messages			Message rates		
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	inventory-release-queue	classic	D	running	0	0	0			
/	inventory-reservation-queue	classic	D	running	0	0	0	0.20/s	0.20/s	0.20/s
/	order-completed-queue	classic	D	running	0	0	0	0.20/s	0.20/s	0.20/s
/	order-failed-queue	classic	D	running	0	0	0			
/	order-saga	classic	D	running	0	0	0	0.60/s	0.60/s	0.60/s
/	payment-process-queue	classic	D	running	0	0	0	0.20/s	0.20/s	0.20/s

Hình 5.3. Các hàng đợi và message Saga trên RabbitMQ Management



5.5. Nhận xét kết quả kiểm thử

Các kịch bản kiểm thử cho thấy hệ thống đáp ứng đúng những yêu cầu nghiệp vụ cốt lõi: định tuyến qua API Gateway, xác thực JWT, luồng mua hàng của khách hàng và đặc biệt là luồng Saga đặt hàng ở cả trường hợp thành công lẫn các trường hợp thất bại có xử lý bù. Khi giữ tồn kho hoặc thanh toán mô phỏng thất bại, hệ thống hủy đơn và hoàn tồn kho đúng như thiết kế, qua đó thể hiện rõ cơ chế nhất quán sau cùng của kiến trúc.

Bên cạnh đó, kiểm thử cũng cho thấy một số hạn chế cần hoàn thiện. Việc giữ tồn kho hiện chỉ kiểm tra số lượng trước khi trừ và chống xử lý lặp theo mã đơn hàng, chưa áp dụng khóa lạc quan ở mức cơ sở dữ liệu, nên trong điều kiện tải cao và nhiều request đồng thời vẫn có khả năng bán vượt tồn kho. Ngoài ra, hệ thống chưa có bộ kiểm thử tự động; đây là những nội dung được đề xuất cải tiến ở chương tiếp theo.

CHƯƠNG 6. ĐÁNH GIÁ VÀ KẾT LUẬN

6.1. Kết quả đạt được

Sau quá trình phân tích, thiết kế, cài đặt và kiểm thử, nhóm đã xây dựng được hệ thống bán cá cảnh FishShop theo kiến trúc microservice với các kết quả chính sau:

- Tách hệ thống thành các service độc lập theo miền nghiệp vụ: API.User, API.Product, API.Order và API.Content.
- Xây dựng API Gateway bằng YARP làm điểm vào chung, định tuyến request, kiểm tra JWT và bổ sung header ngữ cảnh người dùng cùng API key nội bộ.
- Áp dụng nguyên tắc database-per-service: mỗi service quản lý một cơ sở dữ liệu SQL Server riêng, không truy cập trực tiếp dữ liệu của service khác.
- Kết hợp giao tiếp đồng bộ bằng RESTful API và giao tiếp bất đồng bộ bằng RabbitMQ/MassTransit.
- Triển khai Saga orchestration cho luồng đặt hàng: tạo đơn, giữ tồn kho, thanh toán mô phỏng, hoàn tồn kho khi lỗi và cập nhật trạng thái đơn theo cơ chế xử lý bù.
- Tích hợp logging tập trung (Serilog + Seq), tracing (OpenTelemetry + Jaeger) và metrics (Prometheus + Grafana) để theo dõi hệ thống; bổ sung resilience (retry, timeout, circuit breaker) cho các lời gọi liên service.
- Đóng gói backend và hạ tầng quan sát bằng Docker Compose, kèm hai frontend React/Vite cho quản trị và khách hàng.

6.2. So sánh với kiến trúc nguyên khối

So với cách xây dựng theo kiến trúc nguyên khối (monolithic) — đặt toàn bộ chức năng trong một ứng dụng và một cơ sở dữ liệu duy nhất — kiến trúc microservice của FishShop cho phép tách bạch các nhóm nghiệp vụ, phát triển và mở rộng từng phần một cách độc lập, đồng thời giảm ảnh hưởng lan truyền khi một thành phần gặp lỗi. Đổi lại, hệ thống microservice phức tạp hơn trong khâu triển khai, cấu hình và gỡ lỗi do phải vận hành nhiều service, nhiều cơ sở dữ liệu, hàng đợi message và phải xử lý tính nhất quán dữ liệu phân tán. Với một bài toán nhiều module như cửa hàng cá cảnh trực tuyến,

lợi ích về khả năng tách và mở rộng được đánh giá là phù hợp với chi phí phức tạp tăng thêm.

6.3. Ưu điểm

- Loose coupling: các service phụ thuộc lỏng lẻo, giao tiếp qua API và message thay vì dùng chung cơ sở dữ liệu.
- Dễ mở rộng và bảo trì theo từng service, phù hợp bài toán có nhiều module.
- Có khả năng quan sát tốt nhờ logging, tracing và metrics, giúp dễ theo dõi request và message giữa các service.
- Luồng đặt hàng bất đồng bộ bằng Saga giúp giảm phụ thuộc trực tiếp và có cơ chế xử lý bù khi xảy ra lỗi.

6.4. Hạn chế

- Cấu hình và vận hành nhiều service, nhiều cơ sở dữ liệu và hàng đợi tương đối phức tạp so với ứng dụng nguyên khối.
- Phải quản lý tính nhất quán dữ liệu giữa các database; chỉ đạt nhất quán sau cùng chứ không phải nhất quán tức thời.
- Việc giữ tồn kho chưa có khóa lạc quan/bi quan nên còn rủi ro bán vượt tồn kho khi tải cao; cơ chế retry và circuit breaker mới ở mức cơ bản.
- Thanh toán mới ở mức mô phỏng; chưa có bộ kiểm thử tự động; chưa triển khai thực tế trên cloud hoặc Kubernetes.

6.5. Hướng phát triển

- Tích hợp cổng thanh toán thật như VNPAY hoặc MoMo thay cho thanh toán mô phỏng.
- Bổ sung khóa lạc quan cho tồn kho và hoàn thiện cơ chế retry, circuit breaker, idempotency cho message.
- Triển khai hệ thống lên cloud hoặc Kubernetes và xây dựng quy trình CI/CD tự động.
- Tăng cường bảo mật và phân quyền chi tiết hơn (role-based, refresh token, giới hạn truy cập theo tài nguyên).

- Xây dựng bộ kiểm thử tự động (unit, integration) và tối ưu dashboard giám sát trên Grafana.

6.6. Kết luận

Đề tài đã hoàn thành mục tiêu xây dựng hệ thống bán cá cảnh FishShop theo kiến trúc microservice, vận dụng được các kiến thức cốt lõi của môn học Các hệ thống phân tán: tách service theo miền nghiệp vụ, giao tiếp đồng bộ bằng RESTful API và bất đồng bộ bằng message queue, áp dụng database-per-service, Saga cho luồng nghiệp vụ phân tán cùng các cơ chế logging, tracing và monitoring. Hệ thống chạy ổn định trong môi trường demo và thể hiện rõ đặc trưng của hệ thống phân tán. Bên cạnh các kết quả đạt được, nhóm cũng nhận diện được những hạn chế và đề xuất hướng phát triển nhằm đưa hệ thống tiến gần hơn tới một ứng dụng thương mại điện tử thực tế.

TÀI LIỆU THAM KHẢO

- [1] Microsoft, "YARP – Yet Another Reverse Proxy Documentation".
<https://microsoft.github.io/reverse-proxy/>
- [2] Microsoft, "ASP.NET Core documentation".
<https://learn.microsoft.com/aspnet/core/>
- [3] MassTransit, "MassTransit Documentation". <https://masstransit.io/documentation>
- [4] RabbitMQ, "RabbitMQ Documentation". <https://www.rabbitmq.com/docs>
- [5] Microsoft, "SQL Server documentation". <https://learn.microsoft.com/sql/sql-server/>
- [6] OpenTelemetry, "OpenTelemetry Documentation". <https://opentelemetry.io/docs/>
- [7] Serilog, "Serilog – simple.NET logging". <https://serilog.net/>
- [8] Docker, "Docker Compose documentation". <https://docs.docker.com/compose/>
- [9] C. Richardson, "Microservices Patterns", Manning, 2018; Saga pattern.
<https://microservices.io/patterns/data/saga.html>
- [10] .NET / Polly, "Polly resilience library". <https://www.pollydocs.org/>
- [11] Jaeger – <https://www.jaegertracing.io/docs/>; Prometheus –
<https://prometheus.io/docs/>; Grafana – <https://grafana.com/docs/>